

精准控制实验手册

文档版本 01
发布日期 2024-07-05



版权所有 © 华为云计算技术有限公司 2023。 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



HUAWEI和其他华为商标均为华为技术有限公司的商标。
本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为云计算技术有限公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为云计算技术有限公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

精准控制

第一章 实验准备

1.1. 实验描述

本实验的目标是基于 UDP 协议精准控制机器狗的指令动作，准确控制狗的动作、参数等。实验使用的方式包含多种动作，如打招呼、扭身体、太空步、扭身跳等动作，还有对机器狗行走转弯路程的精准控制，同时在昇腾芯片上对多种动作对应的编码指令进行转换并实现相应动作。

1.2. 实验目的

- (1)了解机器狗的运动逻辑、控制逻辑。
- (2)熟悉机器狗的运动指令、运动参数。
- (3)掌握运动数据的建模方式。
- (4)掌握机器狗的 UDP 协议发送。

1.3. 实验建议

本实验主要是用 Python 编程，在开发板上进行运动模型的搭建，调优，最后需要在华为昇腾开发板上进行实际控制。

前置基础课：

- (1) python 高级语法
- (2) 计算机网络
- (3) 机器学习
- (4) 线性回归分析
- (5) 数值优化

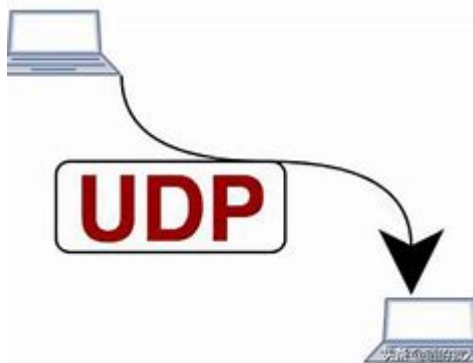
1.4. 实验环境

(前提条件：已经成功烧录 SD 卡)

本次实验使用的硬件包括 PC 机、昇腾 Atlas 200I DK 开发板。实验开始前，请按照如下步骤将各硬件连接：

1. 使用数据线将开发板连接至 PC 机；
2. 给开发版供电，确保其能正常开机；

1.5. 背景知识 1 UDP（用户数据报协议，User Datagram Protocol）



一种无连接的网络传输协议，它属于传输层协议，与 TCP（传输控制协议）并列，但两者在设计 and 用途上有所不同。

UDP 协议的主要特点如下：

- 1.无连接：UDP 是无连接的协议，发送数据之前不需要建立连接。它只是简单地将数据打包到数据报中并发送，不保证数据报的顺序和可靠性。
- 2.简单性: UDP 协议的实现相对简单，因为它没有复杂的控制机制，如 TCP 中的流量控制、拥塞控制和重传机制。
- 3.轻量级: 由于缺乏复杂的控制机制，UDP 的头部开销较小（只有 8 字节），这使得它在传输少量数据时非常高效。
- 4.快速性：UDP 协议的传输速度快，因为它不会等待确认或重传丢失的数据包，适用于对实时性要求较高的应用。
- 5.不保证可靠传输：UDP 不保证数据包的可靠传输，不会对丢失的数据包进行重传，也不会保证数据包的顺序。
- 6.数据报：UDP 使用数据报作为传输单元，每个数据报都是独立的，包含目标端口号和源端口号，以及校验和用于错误检测。
- 7.广播和多播：UDP 支持广播和多播传输，这使得它适用于需要向多个接收者发送相同数据的应用。
- 8.应用场景：UDP 常用于那些可以容忍一定数据丢失，但对传输速度和时延要求较高的应用，如实时视频会议、在线游戏、DNS 查询等。
- 9.校验和：UDP 提供了一个简单的错误检测机制，即校验和，用于检测数据在传输过程中是否出现错误。
- 10.端口号：UDP 使用端口号来区分不同的服务或进程，确保数据能够正确地分发到目标应用程序。

UDP 在机器人中的应用：

- 1.实时控制: 机器人控制通常要求实时响应，UDP 由于其低延迟特性，非常适合实时控制系统。
- 2.传感器数据传输：机器人上的传感器产生的数据可以通过 UDP 快速发送到控制单元进行处理。
- 3.控制命令下发：控制中心或操作员发出的控制命令可以通过 UDP 快速发送到机器人，以实现即时控制。
- 4.多机器人协调: 在多机器人系统中，UDP 可以用于机器人之间的通信，实现协调动作和数据共享。
- 5.简化的通信架构: UDP 的无连接特性简化了通信架构，减少了建立和维护连接的开销。
- 6.容错机制：虽然 UDP 不保证数据的可靠传输，但在机器人控制中，可以通过应用层的设计来实现必要的容错机制,如通过时间戳和序列号来重构数据流。
- 7.网络配置灵活性: UDP 支持广播和多播，使得在配置网络时更加灵活，可以方便地实现一对多或多对多的通信。
- 8.简化的开发: UDP 协议的简单性使得开发者可以快速开发和部署通信代码，加速机器人系统的开发周期。
- 9.与 TCP 的结合使用：在某些情况下，UDP 可以与 TCP 协议结合使用，利用 UDP 的快速性和 TCP 的可靠性来满足不同的通信需求。
- 10.远程控制：在需要远程操作机器人的场景中，UDP 可以用于快速传输操作员的输入到机器人端。
- 11.状态更新: 机器人的状态信息（如位置、速度、电池电量等）可以通过 UDP 定期发送到监控系统。
- 12.紧急停止信号：在紧急情况下，UDP 可以用来快速发送停止信号到机器人，以确保安全。

1.6. 背景知识 2 机器狗运动正向求解

运动方程求解通常是指在已知物体的初始条件和作用力的情况下，使用物理定律来确定物体的运动状态（如位置、速度、加速度）的过程。以下是求解运动方程的一般步骤：

a. 确定已知条件

- **初始条件：** 物体的初始位置、初始速度、初始加速度等。

- **作用力**：作用在物体上的力，包括重力、摩擦力、外力等。

b. 选择合适的物理定律

- **牛顿运动定律**：适用于宏观物体的经典力学问题。
- **拉格朗日方程**：适用于更复杂的系统，如多体系统或存在约束的系统。
- **哈密顿方程**：适用于需要考虑能量守恒和动量守恒的系统。

c. 建立运动方程

- 根据牛顿第二定律， $F=ma$ ，可以推导出加速度。
- 对于更复杂的系统，需要使用拉格朗日量 $L=T-V$ 或哈密顿量 $H=T+V$ 来推导运动方程。

d. 求解方程

- **解析求解**：如果方程可以解析求解，直接求解得到物体的运动状态。
- **数值求解**：对于复杂的方程，需要使用数值方法，如欧拉法、龙格-库塔法等。

e. 考虑约束条件

- 如果系统存在约束，如铰链、滑轮系统等，需要在求解过程中考虑这些约束。

f. 模拟和验证

- 使用计算机模拟求解结果，与理论预期或实验数据进行比较，验证模型的准确性。

g. 分析结果

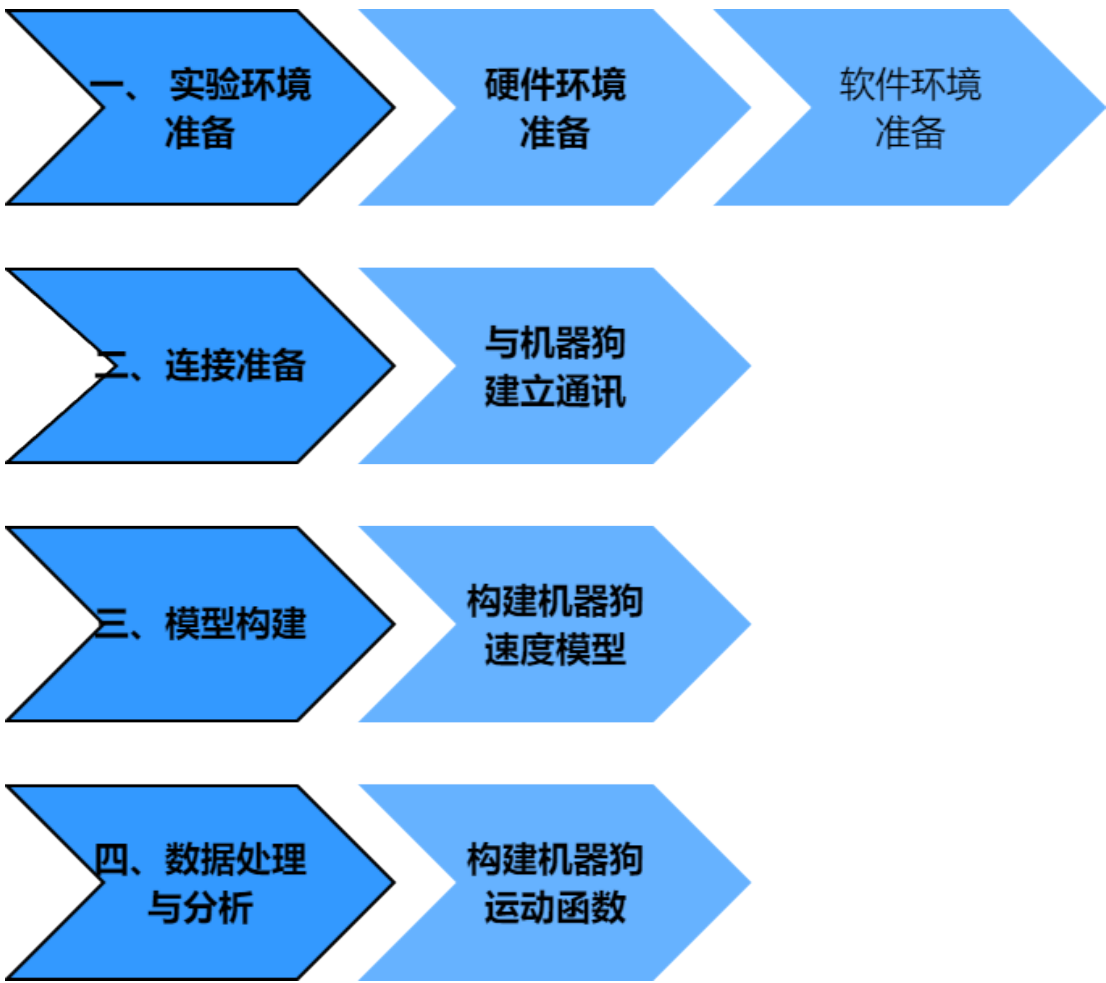
- 分析求解得到的运动状态，如物体的轨迹、速度变化等。

第二章 实验过程简述

本项目的主要流程为：

与机器建立通讯→构建机器狗速度模型→构建运动函数

本实验分成如下步骤完成：



第三章 实验环境准备

3.1. 硬件环境准备

(1) SD 卡中系统烧录

向 SD 卡中烧录镜像的步骤如下。

① 将 SD 卡插入读卡器，再将读卡器插到 PC 上。



② 在浏览器中进入 [Ascend 官网](#)，选择 “产品” -> “开发者套件”



③ 点击 “资源”，进入资源下载界面



④ 选择 “制卡工具下载”，下载完成后进行安装。

寻你所需，玩转AI开发_

快速入门

准备

请提前准备好下列物品，或查看 [硬件准备清单](#)，快速上手不卡顿

- MicroSD卡（推荐64GB）及读卡器（可选）
- 获取制卡工具，一键烧录镜像完成开发环境准备

[↓ 制卡工具下载](#) [↓ 校验文件下载](#)

入门

提供入门 Atlas 200I DK A2 的全流程指导，帮助你30分钟内轻松完成从启动开发者套件到运行首样例的过程

[▶ 快速开始（课程）](#) [📄 快速开始（文档）](#)

参考工具

提供Atlas 200I DK A2 进一步开发所需的参考工具及配套资源，可按需下载

模型适配工具

使用开发者0代码完成数据标注、模型训练、模型部署到昇腾开发者套件

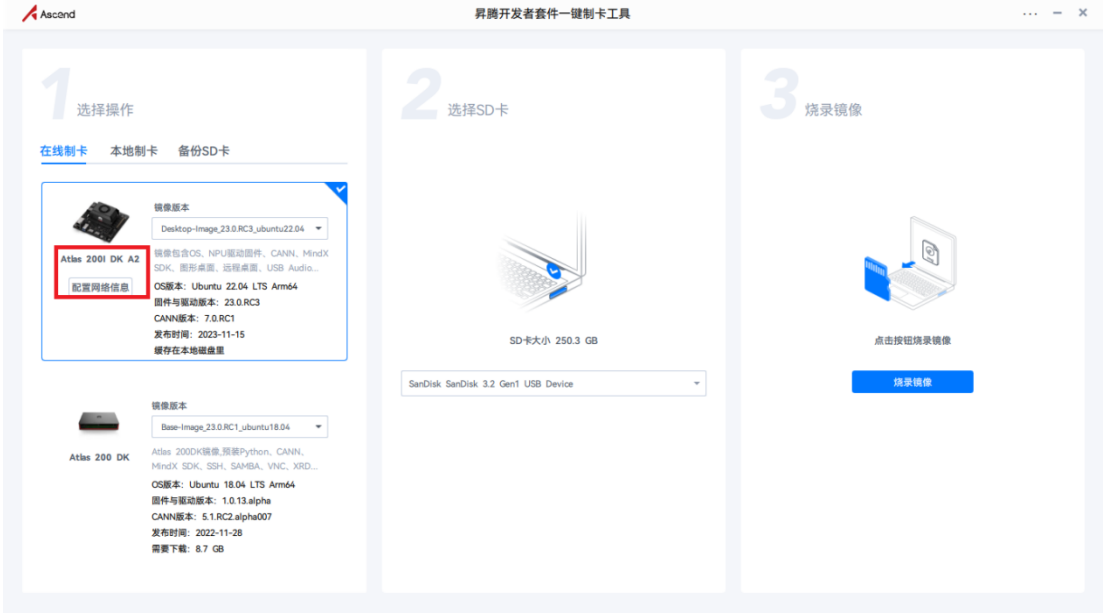
[↓ 软件包下载](#) [↓ 校验文件下载](#)

配套Conda环境包

模型适配工具运行所必须的配套conda依赖环境，需要提前安装conda并导入conda环境包

[↓ 软件包下载](#) [↓ 校验文件下载](#)

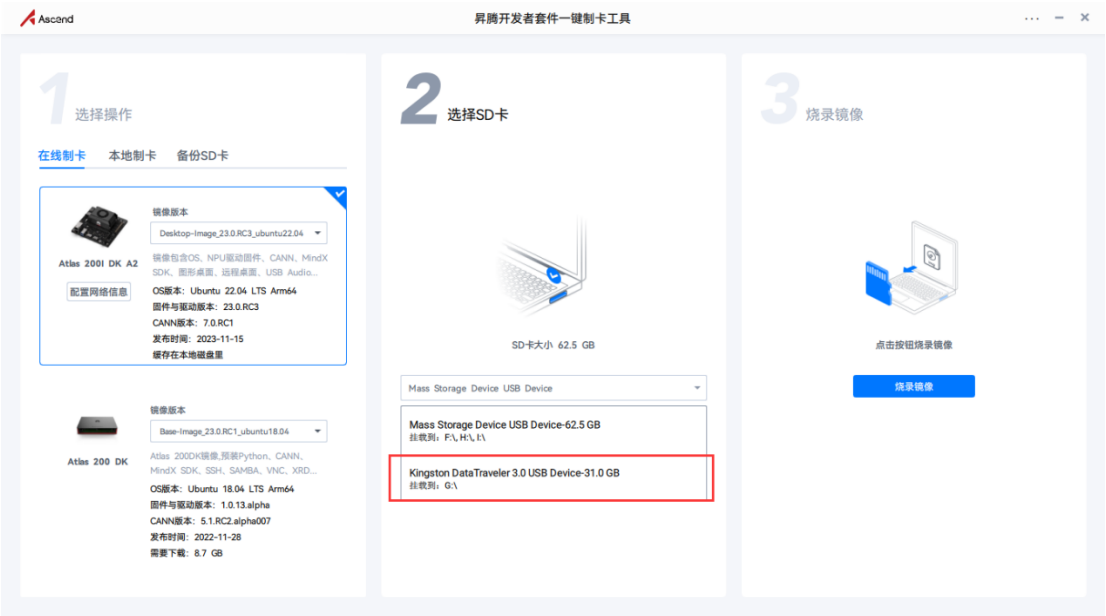
⑤ 打开镜像烧录工具 Ascend AI Devkit Imager，选择 Altas 200I DK A2



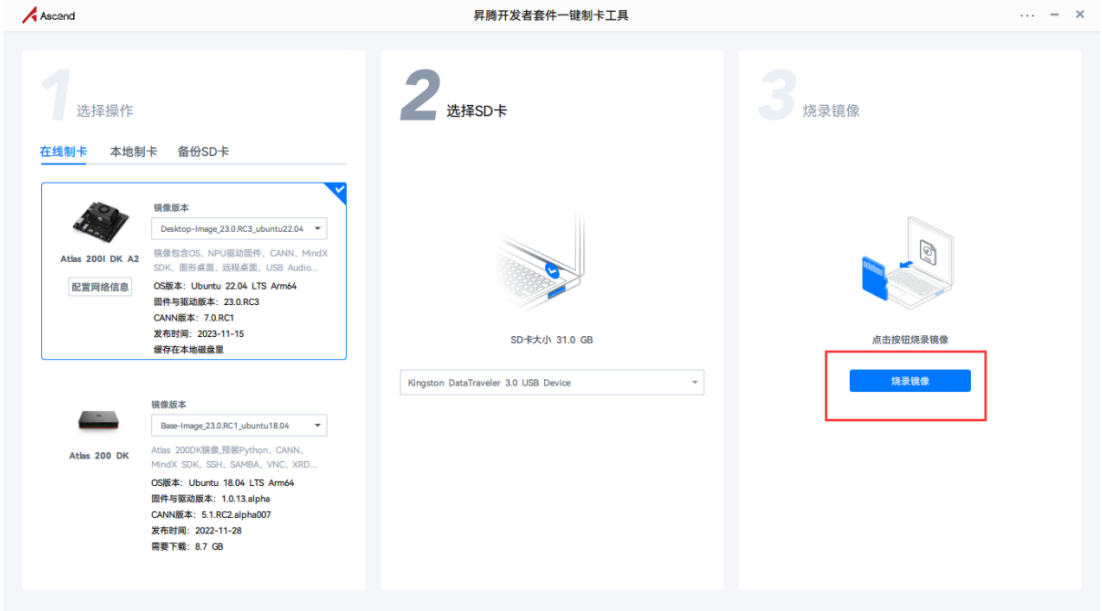
⑥打开“配置网络信息”，选择“Type-C”选项卡。可以看到 Type-C 接口默认 IP 为 192.168.0.2。选择默认 IP。



⑦选择要烧录的 SD 卡

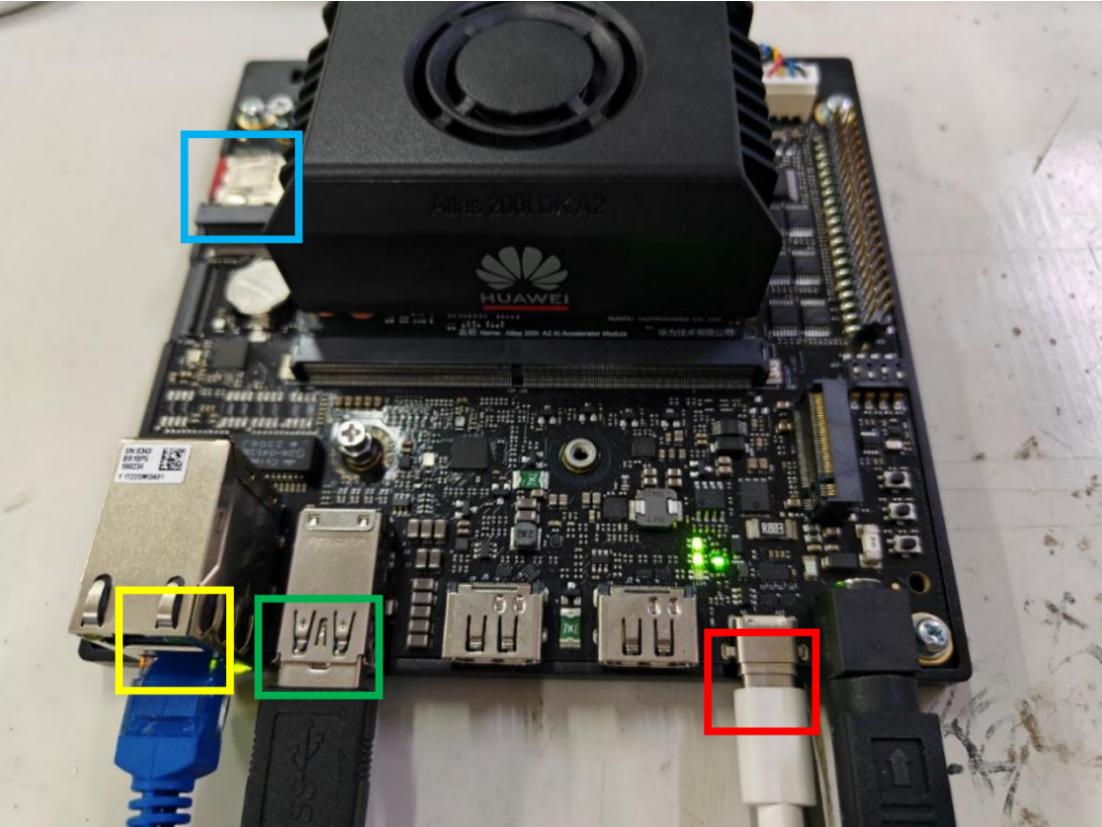


⑧点击“烧录镜像”，等待镜像烧录完成。



(2) 开发板连接

将烧录好系统的 SD 卡插入到开发板中（下方蓝色框中），昇腾开发板和 PC 机通过 Type-C 线连接，将 Type-C 线和电源线连接到开发板（下方红色框中），并将海康工业相机和机器狗的 USB 线连接到开发板（下方绿色框中）。同时可以连接网线（下方黄色框中），让开发版连接互联网，方便环境依赖包的下载。连接方式如下图所示。



(3) PC 端远程连接开发板

本节介绍当开发者套件通过 Type-C 接口和 PC 连接时,如何在 PC 将接口 IP 地址修改为和开发者套件接口 IP 地址同一个网段（如开发者套件 Type-C 接口为 192.168.0.2，PC 对应 USB 或 Type-C 接口为 192.168.0.101），并使用 SSH

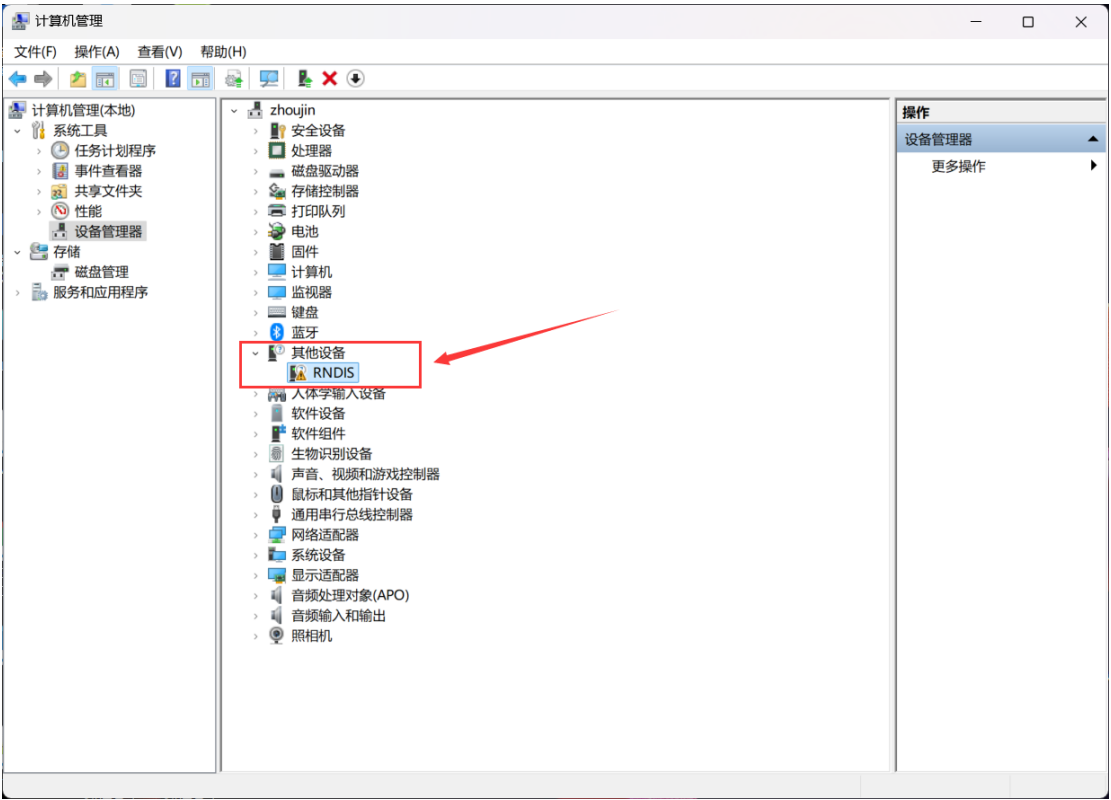
工具远程登录开发者套件。

1) 安装 Windows 的 USB 网卡驱动

本步骤以 Windows 10 系统为例。

①在 PC 上右键单击“此电脑”，选择 “更多” → “管理”，进入“计算机管理”界面。

②在“计算机管理”操作界面中选择“设备管理器→ “其他设备”，如图 2 所示，RNDIS 为未识别状态。



注意：如果未出现 RNDIS，请按以下方法排查：

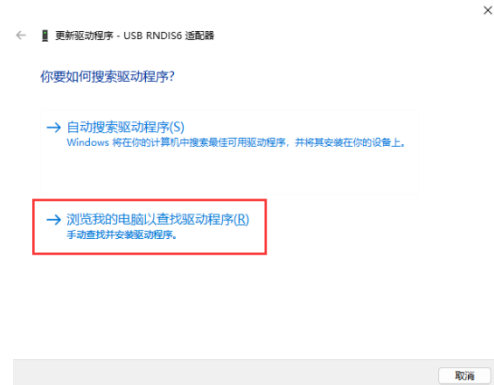
确保开发者套件 Type-C 接口和 PC 已通过连接线正常连接。

更换具备数据传输能力的 Type-C 数据线（一般手机充电线拥有数据传输能力），继续查看是否出现 RNDIS。

如果尝试现场所有 Type-C 数据线，仍无法出现 RNDIS，则考虑使用 RJ45 网线连接 PC 和开发者套件的以太网口的方式登录开发者套件。

③右键单击 “RNDIS”，选择 “更新驱动程序(P)”。

④在弹出的 “更新驱动程序” → “RNDIS 窗口” 中选择 “浏览我的计算机以查找驱动程序软件 (R)”。



⑤然后选择 “让我从计算机上的可用驱动程序列表中选择(L)”。



⑥在 “常见硬件类型” 列表中选择 “网络适配器”，单击 “下一页(N)”。



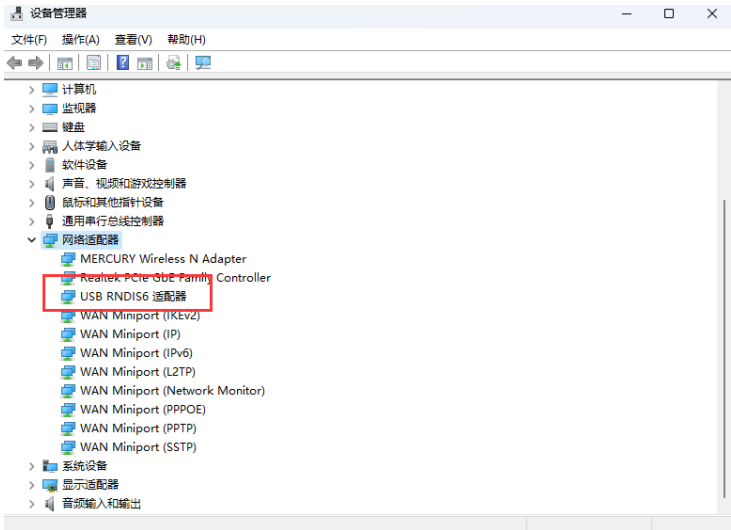
⑦在 “选择要为此硬件安装的设备驱动程序” 界面中选择 “Microsoft” 厂商的 “USB RNDIS6 适配器”。



⑧单击“下一页”，在弹出的“更新驱动程序警告”窗口选择“是”。

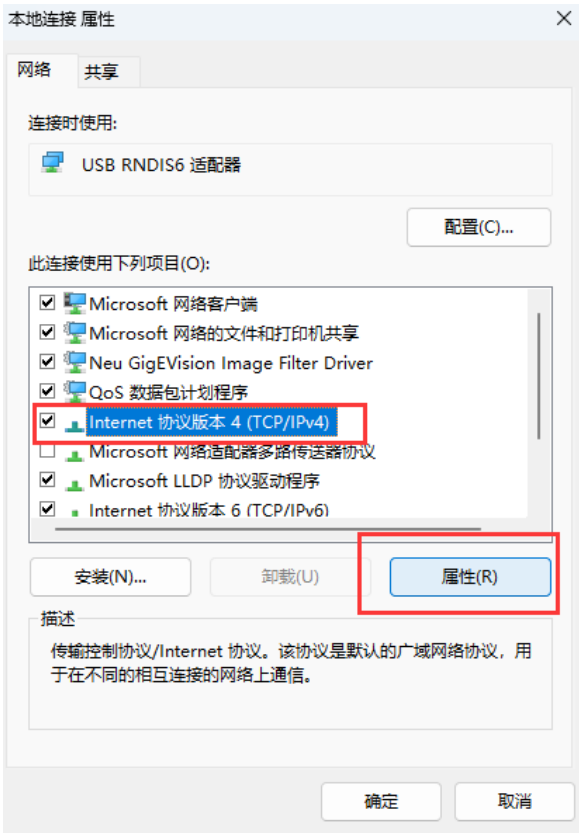
⑨返回设备管理器”→ “网络适配器”，可看到已经正常显示了 USB RNDIS6

适配器的驱动。



(4) 配置 PC 接口 IP 地址

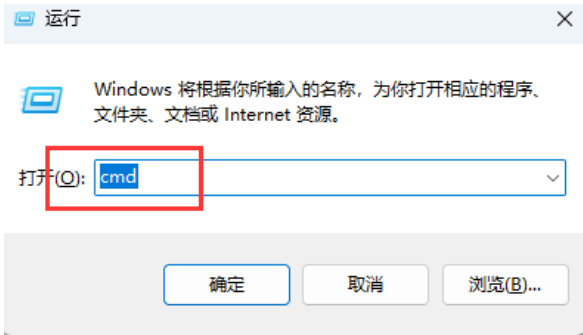
- 本步骤以 Windows 10 系统为例。
- ①在 PC 上打开“开始”，单击“设置”按钮，进入“Windows 设置”界面。
 - ②选择“网络和 Internet”，单击“更改适配器选项”。
 - ③鼠标右键单击“本地连接”后鼠标左键单击“属性”进入“本地连接 属性”界面（使用 Type-C 接口连接时一般为“本地连接 x”，x 为数字，以实际显示的数字为准。
 - ④选择“Internet 协议版本 4 (TCP/IPv4)”，单击“属性”。



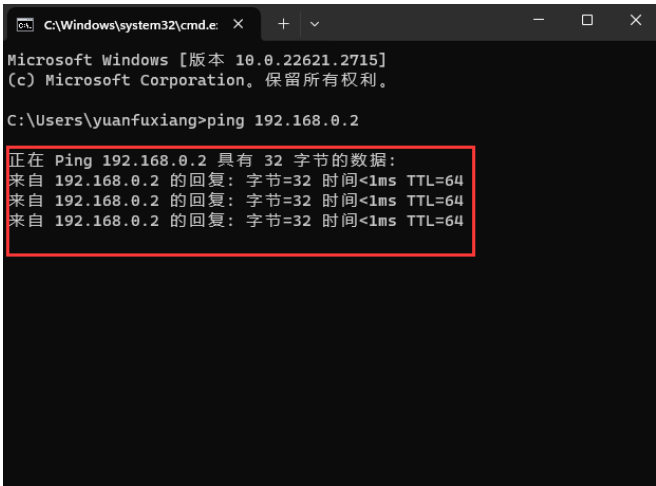
- ⑤勾选“使用下面的 IP 地址”选项，填写 IP 地址（图示以 192.168.0.101 为例）和子网掩码，默认网关与 DNS 服务器地址为空，单击“确定”保存。



⑥使用快捷键 “Win+R”，在运行窗口输入 cmd 进入命令行窗口。



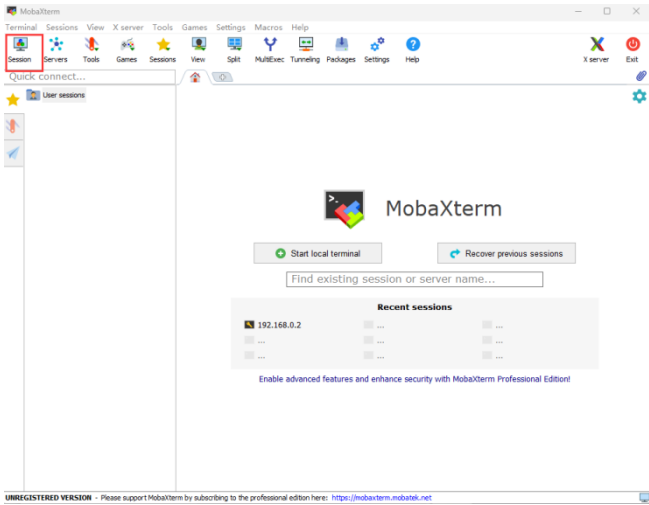
⑦输入 ping 192.168.0.2 命令，查看是否能够访问开发板 IP。



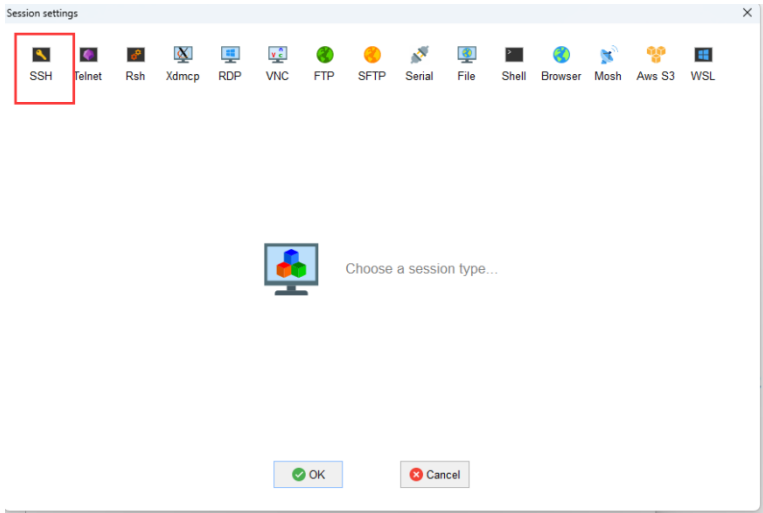
(5) 使用 ssh 远程登录开发板

本文使用 MobaXterm 工具在 PC 上远程登录开发板，单击[下载链接](#)获取 MobaXterm 软件压缩包，解压获得“MobaXterm_Personal_22.2.exe”。

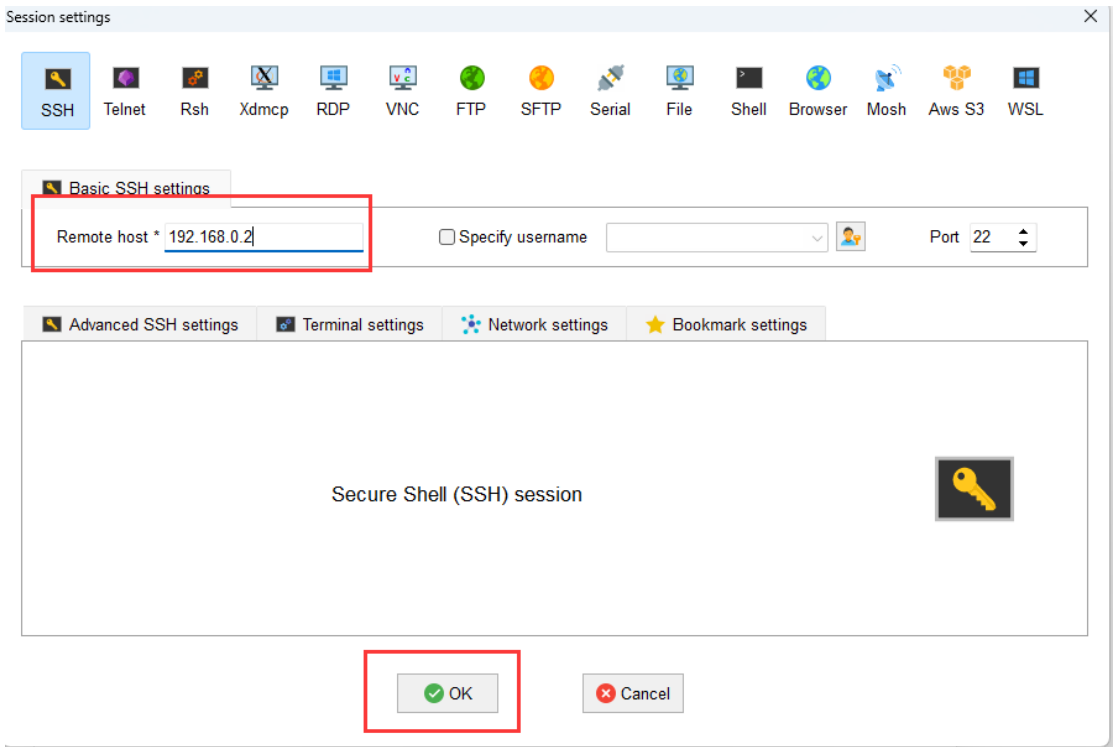
- ①双击 “MobaXterm_Personal_22.2.exe” 启动 SSH 登录工具。
- ②单击左上方的 “Session” 进入界面。



③单击左上方的“SSH”进入 SSH 连接配置界面



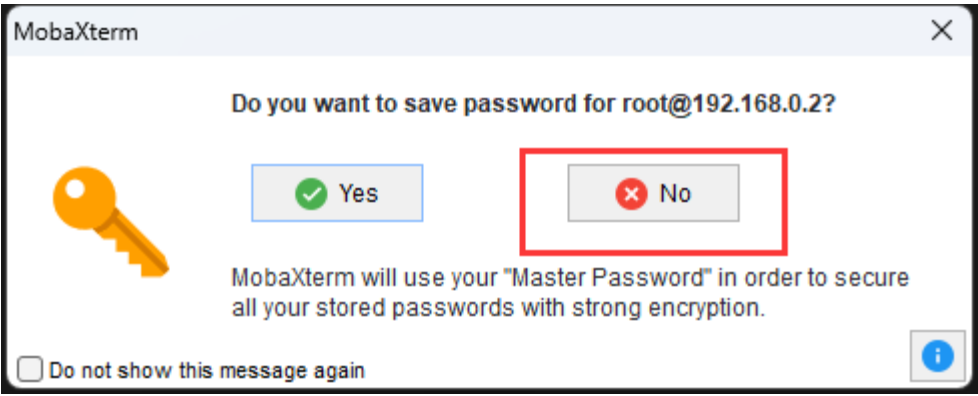
④根据硬件连接方式填写开发者套件连接 PC 的接口实际 IP 地址（一键制卡中配置的 IP 地址）。图示以 192.168.0.2 为例，请根据实际修改。



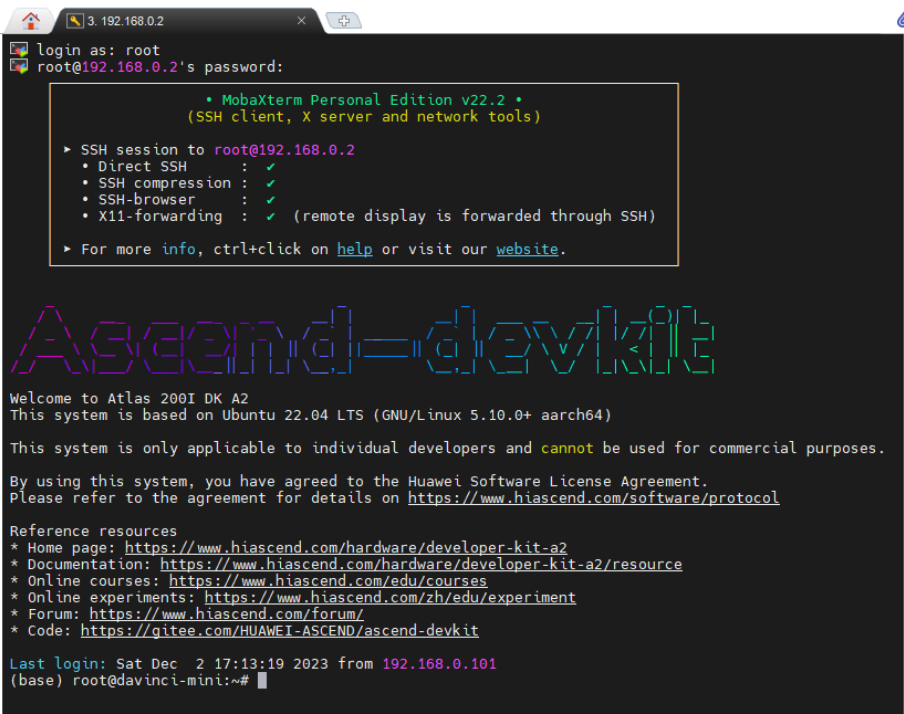
⑤单击“OK”按钮，首次连接开发者套件时，SSH 工具提示是否信任连接的设备，单击“Accept”。



⑥进入远程登录界面后，输入 root 用户名登录密码（默认为 Mind@123）登录开发者 套件，请修改默认密码，并妥善保管新密码。 SSH 工具界面会出现保存密码提示，可以单击“No”，不保存密码直接登录开发者套件。



⑦远程登录开发者套件成功界面如图所示。



(6) 机器狗中继连接外网

注: 本教程主要为暂时没有外置无线网卡的用户提供连接外网方案，教程以中继手机热点为例进行说明，中继无线路由器同理。

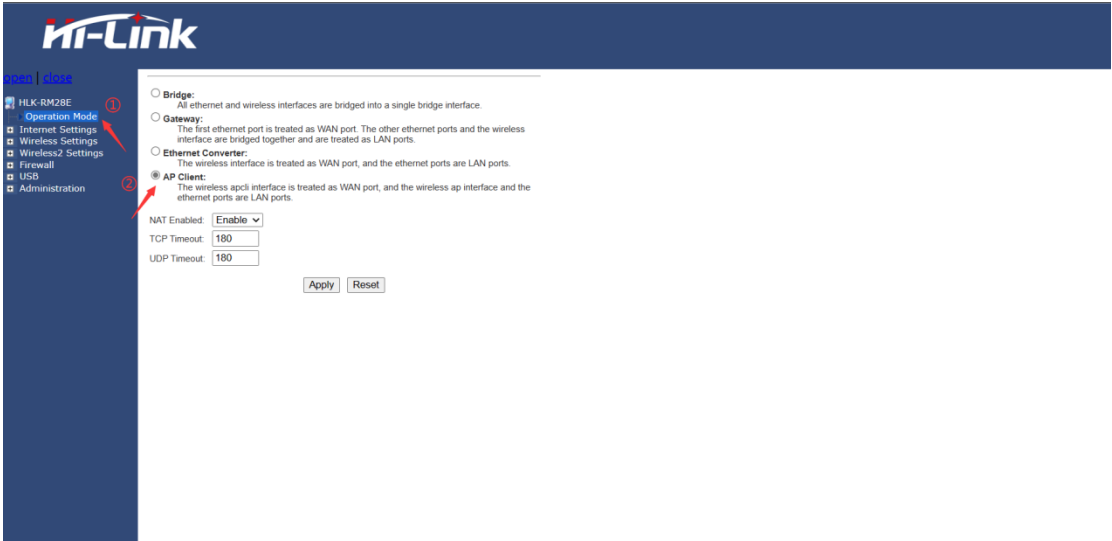
1、电脑连接 机器狗的 wifi, 点击电脑右下角的 wifi 图标, 点击属性查看 IPv4 DNS 服务器的 ip 地址。如下图所示。



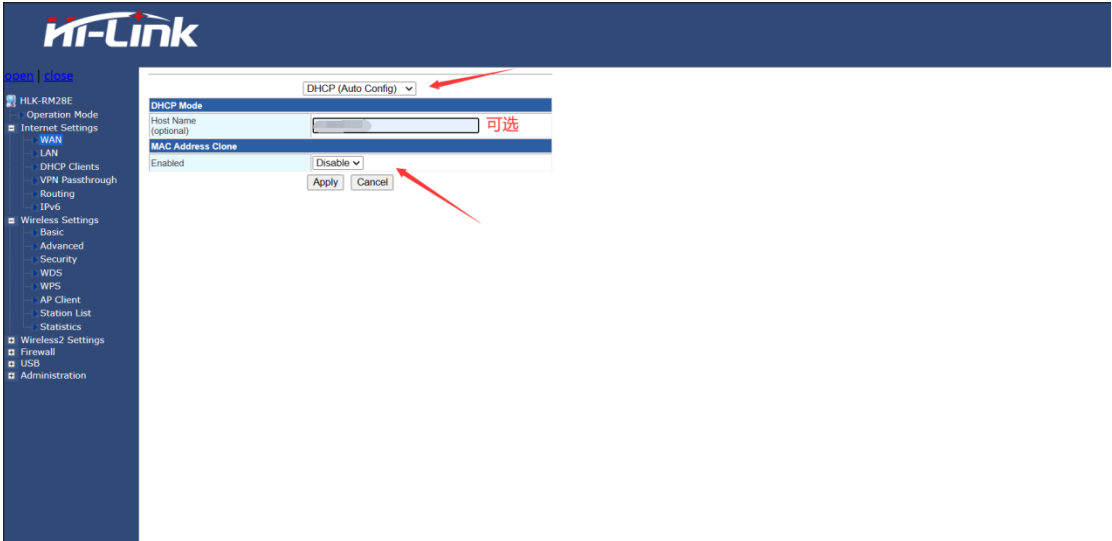
2、打开电脑浏览器，输入 IPv4 DNS 服务器的 ip，默认登录用户和密码均为 admin。如下图所示。



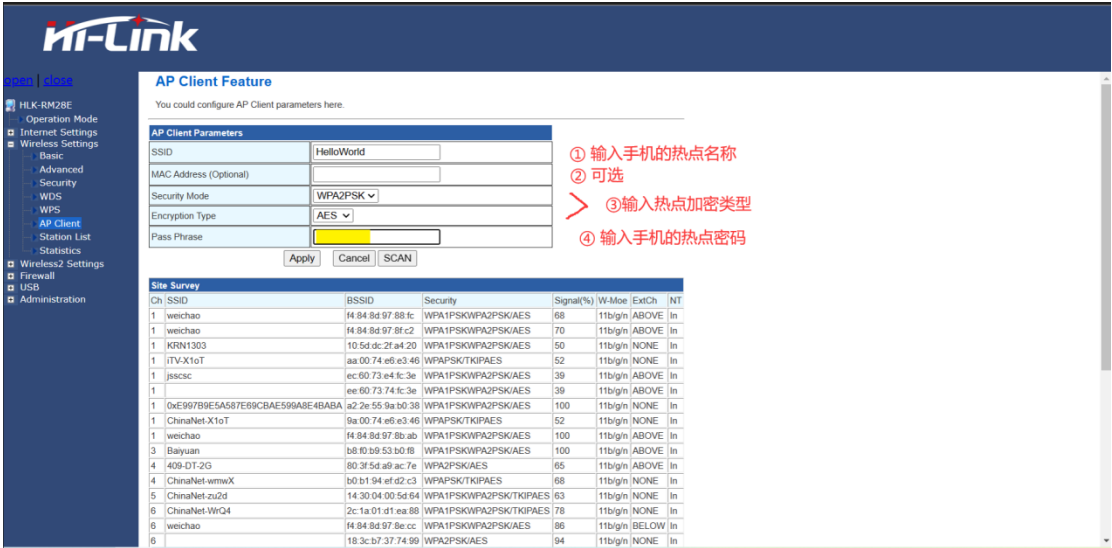
- 3、打开手机热点。
- 4、进入 Hi-link 设置页面，点击 Operation Mode，选择 AP Client，最后点击 Apply。如下图所示。



5、展开 Internet Settings，点击 WAN，参照下图进行设置，最后点击 Apply。



6、展开 Wireless Settings，点击 AP Client，查看 site Survey 表中是否有自己的手机热点名称，如若没有点击 SCAN 进行扫描，在 AP client Parameters 表中参照下图进行设置，最后点击 Apply。



7、重启机器狗，注意中继时热点需一直处于开启状态，电脑连接机器狗的wifi，使用远程软件登录，此时机器狗处于连接外网状态。测试是否有网络，点击浏览器，搜索任意内容，如“百度学术”，能正常显示下图说明正常。



3.2. 机器狗远程连接

用户可通过 SSH 远程连接到运动主机。

1. 将开发主机连接到机器人 WiFi。
2. 在开发主机上打开 SSH 连接软件，输入 `ssh firefly@192.168.1.120`，密码为 `firefly`，即可远程连接运动主机；
3. 进入 `jy_exe`，并打开配置文件：

```
cd /home/firefly/jy_exe/conf/
vim network.toml
```

4. 配置文件 `network.toml` 内容如下：

```
ip = '192.168.1.102'

target_port = 43897

local_port = 43893

~
```

- 5.修改配置文件第一行中的 IP 地址，使得代码能够接收到机器狗数据：
 - 如果代码在机器人运动主机内运行，IP 设置为运动主机 IP：192.168.1.120；
 - 如果代码在用户自己的开发主机中运行，设置为开发主机的静态 IP：192.168.1.xxx；
6. 重启运动程序使配置生效。

```
cd /home/firefly/jy_exe
sudo ./stop.sh
```

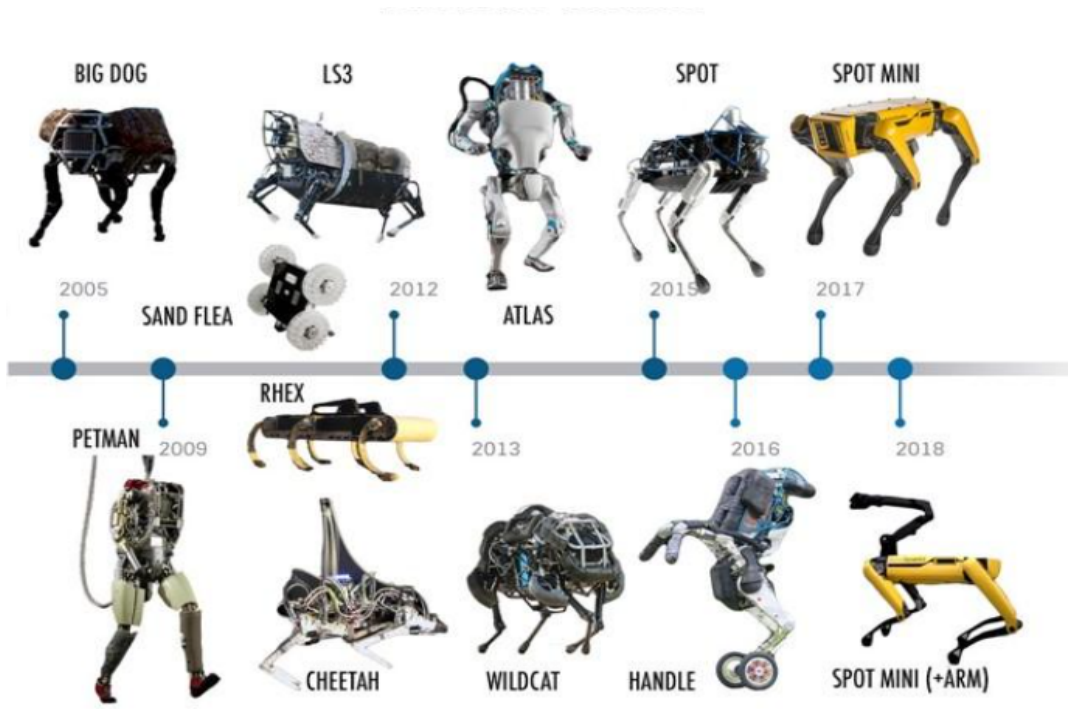
Error! Unknown document property name.

```
| sudo ./restart
```

第四章 机器狗介绍

4.1. 研究背景与意义

随着科技进步，社会生产力迅猛发展，机器人行业得到了蓬勃发展，其中四足机器人发展迅速，被广泛应用。传统的机器人如轮式、履带式，虽然具有很快的移动速度，但受地形约束。四足机器人是一种仿生机器人，在跨越地形障碍方面具有很大优势，足式移动机器人对行走路面的要求很低，它可以跨越障碍物，在各种崎岖不平的复杂路面上行进。在户外，车辆或履带式机器人无法应对大部分的复杂崎岖地形，四足式机器人以其灵活机动的特性在野外勘测和信息传递方面发挥重要的作用。



未来，四足机器人的发展趋势主要有以下四个方向。

- 增加腿部的自由度，提高机动性和灵活性
- 提高四足机器人的稳定性
- 增强四足机器人的负重能力和行进速度

- 与人工智能相结合

当代人工智能的发展如火如荼，将人工智能与四足机器人相结合，让四足机器人模仿人类的学习能力，使机器人具备判断和决策能力，与不同的机器人之间相互协作，实现智能化。使机器人能适应复杂的环境，完成更加复杂的任务，与 AI 结合也是一个重要的研究方向。更智能的四足机器人或将成为导盲四足机器人，为残障人士的出行和安全提供极大的便利和保障。

第五章 工程文件目录结构

5.1. Ascend

本章节主要展示的是在开发板上所有的文件，以及每个文件所对应的作用。本章节主要展示开发板上所有的文件及其对应的作用，以便更好地理解整个项目的结构和功能。

```
(base) root@davinci-mini:/opt/lab/PreciseControl# tree -I "__pycache__|*.pyc" -L 10
.
├── command
│   ├── __init__.py
│   └── udp_command.py
├── precise_control_main.py
├── sendcommand
│   ├── SendToCommand.py
│   ├── __init__.py
│   └── heartbeat.py
├── socketnetwork
│   ├── __init__.py
│   └── network_utils.py
├── speeds
│   ├── __init__.py
│   └── sportsspeed.py
└── threading_utils
    └── ThreadTemplates.py

5 directories, 11 files
```

开发板的文件夹总名为‘PreciseControl’，如图所示。Precise_control_main.py是主程序，该文件夹包含多个子文件夹和文件，每个文件都有其特定的用途。下面是各个文件夹及其主要文件的详细介绍：

Command 文件夹

- **udp_command.py**: 该文件用于存放指令码。它定义了各种控制命令，这些命令通过UDP协议发送到机器狗，以实现对其精确控制。

SendCommand 文件夹

- **heartbeat.py**: 该文件用于发送心跳包。心跳包是一种定期发送的信号，用于检测系统或设备是否在线和正常工作。通过发送心跳包，可以确保机器狗与控制系统之间的通信连接是稳定和可靠的。
- **SendToCommand.py**: 该文件负责发送指令。它包含了一系列函数，用于将指令发送到机器狗，确保指令能够正确地传达和执行。

SocketNetwork 文件夹

- **network_utils.py**: 该文件是与机器狗建立通信的脚本。它包含了各种网络通信的实用函数,用于初始化和维护与机器狗的通信连接,使得控制指令能够通过网络传输到机器狗。

Speeds 文件夹

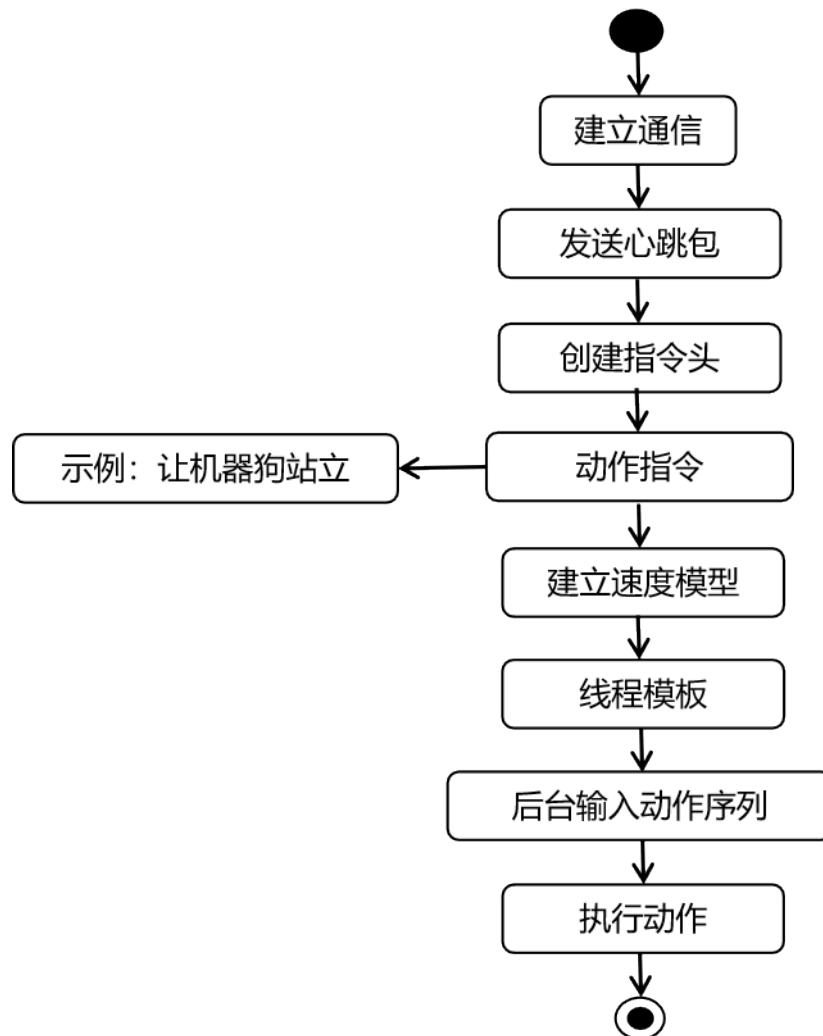
- **sportspeed.py**: 该文件定义了机器狗运动的速度方程。通过调整速度参数,可以控制机器狗的运动速度,使其能够按照预定的轨迹和速度执行任务。

ThreadingUtils 文件夹

- **ThreadTemplates.py**: 该文件定义了线程模板。线程是并发执行的基本单位,通过使用线程,可以提高系统的响应速度和处理效率。该文件中的模板可以用于创建和管理多个线程,以实现并行处理。

这些文件共同构成了‘PreciseControl’项目的核心,通过合理的模块划分和功能实现,使得项目能够高效、稳定地运行。在实际开发和调试过程中,了解每个文件的具体作用,有助于快速定位问题并进行相应的修改和优化。

第六章 精准控制



6.1. 建立通信

简单控制第一步：建立通信链接， `socketnetwork/network_utils.py` 文件定义了一个函数 `setup_socket_and_address`，用于创建一个 UDP 套接字，并设置机器狗的 IP 地址和端口号作为目标地址。该通信链接是所有后续控制和数据传输的基础。不仅仅确保了指令传输的可靠性和及时性，还实现对机器狗的远程控制和监控以及为数据传输提供通道，代码如下：

```
1. import socket
2. from typing import *
```

```

3.     def setup_socket_and_address(dest_ip = '192.168.1.120', port=43893) -> Tuple[socket.socket, Tuple[str, int]]:
4.         # 创建UDP 套接字
5.         sfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
6.         # 设置目标地址
7.         target_address = (dest_ip, port)
8.         # print(target_address)
9.         return sfd, target_address

```

这段 Python 代码定义了一个函数 `setup_socket_and_address`，其目的是设置一个 UDP 套接字并返回该套接字及其目标地址。下面是对代码的逐行分析：

`import socket` - 导入 Python 的 `socket` 库，这个库提供了访问底层网络接口的机制。

`from typing import *` - 从 Python 的 `typing` 模块导入所有类型提示，用于在函数和方法中添加类型注解，以提高代码的可读性和健壮性。

`def setup_socket_and_address(dest_ip = '192.168.1.120', port=43893) -> Tuple[socket.socket, Tuple[str, int]]:` - 定义了一个名为 `setup_socket_and_address` 的函数，它接受两个参数：`dest_ip` 和 `port`，这两个参数分别指定了目标 IP 地址和端口号，默认值分别是 `'192.168.1.120'` 和 `43893`。函数的返回类型被注解为 `Tuple[socket.socket, Tuple[str, int]]`，意味着它将返回一个元组，其中包含一个 `socket` 对象和一个包含 IP 地址和端口号的元组。

`sfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)` - 创建一个新的 UDP 套接字。`socket.AF_INET` 表示使用 IPv4 地址，`socket.SOCK_DGRAM` 表示使用 UDP 协议。

`target_address = (dest_ip, port)` - 将传入的 IP 地址和端口号组合成一个元组，

这个元组将用作套接字的目标地址。

6.2. 发送心跳包

心跳包是一种周期性发送的信号，用于确保网络连接的活跃状态，防止因长时间无通信而被网络设备断开连接。这对于实时控制和数据传输至关重要。确保系统在运行过程中不会因为通信问题而出现故障，提高系统的整体可靠性。发送心跳包代码位于 `sendcommand/heartbeat.py`，代码如下：

```
1.  #!/usr/bin/env python
2.  # -*- coding: utf-8 -*-
3.  import socket
4.  import struct
5.  import time
6.
7.  def send_udp_heartbeat(sfd, target_address, code=0x210400
01, parameters_size=0, type=0, heartbeat_interval=0.25) -> Non
e:
8.      # 打包心跳指令数据
9.      heartbeat_command = struct.pack('<III', code, paramet
ers_size, type)
10.
11.     try:
12.         while True:
13.             # 记录发送前的时间
14.             start_time = time.time()
15.
16.             # 发送心跳包
17.             sfd.sendto(heartbeat_command, target_address)
18.
```

```

19.             # 发送后的延时
20.             time.sleep(max(0, heartbeat_interval -
21.                             (time.time() - start_time)))
22.         except KeyboardInterrupt:
23.             print("Heartbeat sending stopped by user.")
24.         finally:
25.             # 关闭 socket
26.             sfd.close()

```

`network_utils.setup_socket_and_address()` 函数被调用，用于创建一个 UDP 套接字（或其他类型的网络通信接口）并设置机器狗的 IP 地址和端口号作为目标地址。导入所需的模块: `socket` 用于网络通信，`struct` 用于数据打包，`time` 用于时间相关的操作。

定义了 `send_udp_heartbeat` 函数，它接收以下参数：

`sfd`: 一个 `socket` 对象，用于发送数据。

`target_address`: 目标地址的元组，包含 IP 地址和端口号。

`code`: 心跳指令的代码，默认为 `0x21040001`。

`parameters_size`: 参数大小，默认为 `0`。

`type`: 心跳包的类型，默认为 `0`。

`heartbeat_interval`: 心跳包发送间隔，默认为 `0.25` 秒。

使用 `struct.pack` 将心跳指令的数据打包成字节串。参数 `<III` 表示使用小端格式打包三个整数。

10-22. 在一个无限循环中，执行以下操作：

记录发送前的时间（第 15 行）。

发送心跳包到目标地址（第 18 行）。

计算发送后需要等待的时间，以确保心跳包的发送间隔符合 `heartbeat_interval`（第 21 行）。

23-24. 使用 `try-except` 结构来捕获 `KeyboardInterrupt` 异常，这通常发生在用户使用 `Ctrl+C` 中断程序时。如果捕获到这个异常，会打印一条消息表明心跳发送已被用户停止。

25-27. 使用 `finally` 块来确保无论是否发生异常，`socket` 都会在函数结束时关闭。

启动心跳线程，代码如下：

```
1.  # 建立通讯
2.  sfd, target_address = network_utils.setup_socket_and_address()
3.
4.  # 定义发送心跳包的线程执行体
5.  # 注意*args 必须以tuple形式传入，包括sfd和target_address
6.  heartbeat_thread = MyRepeatThread("HeartbeatThread", heartbeat.send_udp_heartbeat, 0.25, None, sfd, target_address)
7.  heartbeat_thread.start()
```

`heartbeat_thread.start()` 启动了心跳线程，它将按照设定的时间间隔（这里是 0.25 秒）发送心跳包（`MyRepeatThread` 在后文中会解释）。

6.3. 创建指令头

创建指令头主要为了构建通信协议，提高代码的可读性和可维护性以及支持复杂的指令操作。指令头包含了关键信息，如指令码、参数大小和类型，是通信协议的核心组成部分。通过定义指令头，可以构建和解析复杂的通信协议。

1. #使用示例（创建指令头）
2. **action_CommandHead** = CommandHead()

创建了一个 **CommandHead** 对象，这个对象包含了指令的头部信息，如动作代码、序列号等。

```
1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3.
4. # 导入 ctypes 模块，用于操作 C 类型数据
5. import ctypes
6. # 导入 struct 模块，用于处理二进制数据包
7. import struct
8. # 导入 inspect 模块，用于获取类和其他对象的信息
9.
10. # 定义 CommandHead 类，用于表示指令的头部信息
11. class CommandHead:
12.     # CommandHead 类的初始化方法
13.     def __init__(self, code=0, parameters_size=0, type_=0):
14.         # 指令码，用于标识指令
15.         self.code = code
16.         # 指令参数大小，表示指令参数所占的字节数
17.         self.parameters_size = parameters_size
18.         # 指令类型，用于区分不同类型的指令
19.         self.type_ = type_
20.
```

```

21. # 定义 Command 类, 用于表示一个完整的指令, 包括头部和数据
22. class Command:
23.     # 指令数据的大小常量, 设为 256 字节
24.     kDataSize = 256
25.
26.     # Command 类的初始化方法
27.     def __init__(self, head=None, data=None):
28.         # 如果没有提供指令头, 则创建一个默认的 CommandHead 对
           象
29.         if head is None:
30.             head = CommandHead()
31.         # 如果没有提供数据, 则创建一个大小为 kDataSize 的全 0
           列表
32.         if data is None:
33.             data = [0] * self.kDataSize
34.         # 将指令头和数据保存到实例变量
35.         self.head = head
36.         self.data = data

```

定义了两个类, `CommandHead` 和 `Command`, 它们用于构建和处理通信协议。

下面是对这些类的简要解释:

`CommandHead` 类: 这个类用于表示命令的头部信息, 包含三个属性。

`code`: 指令码, 通常用于标识具体的命令。

`parameters_size`: 指令值的大小, 表示参数的长度。

`type_`: 指令类型, 用于区分命令的种类或优先级。

`Command` 类: 这个类用于表示一个完整的命令, 它包含头部信息和数据部分。

`kDataSize`: 类属性, 定义了数据部分的最大大小, 这里设置为 256。

构造函数接受两个参数:

`head`: 命令头对象, 默认为 `None`, 如果没有提供, 将创建一个 `CommandHead` 实例。

`data`: 命令的数据部分, 默认为一个包含 256 个 0 的列表, 如果没有提供, 将使用这个默认值。

6.4. 动作指令

动作指令目的是调用机器狗的具体动作, 通过将动作代码传入结构体中, 并通过函数发送指令到机器狗, 实现远程控制

```
1. # 调用机器狗动作
2.
3. # 设置 action_CommandHead 的 code 属性为 dog_actions 字典中
   "ACTION_STAND_DOWN" 对应的值
4. # action_CommandHead 是一个 CommandHead 类的实例, 用于存储动作
   的头部信息
5. # dog_actions 是一个字典, 包含了不同动作的标识码
6. action_CommandHead.code = dog_actions.get("ACTION_STAND_DOW
   N")
7.
8. # 使用 SendToCommand 类的 perform_action 方法发送动作命令
9. # sfd 是一个套接字文件描述符, 用于网络通信
10. # target_address 是目标地址, 包含了 IP 地址和端口号
11. # action_CommandHead.code 是要发送的动作命令的代码
```

12. `# 0, 0` 是动作命令的额外参数, 这里传入了 `0`

```
13. SendToCommand.perform_action(sfd, target_address, action_
    CommandHead.code, 0, 0)
```

`SendToCommand.perform_action()` 函数用于发送具体的指令到机器狗。里面的参数 `sfd` 是套接字文件描述符, `target_address` 是机器狗的网络地址, `action_CommandHead.code` 是动作代码。

其中 `dog_actions` 的作用是获取到指令码, 指令码已经以字典的形式放在了 `command/udp_command.py` 中。

```
1.  dog_actions = {
2.      # 简单指令
3.      "ACTION_STAND_DOWN": 0x21010202,          #
   起立/趴下 0x21010202 - 0 在趴下状态和初始站立状态之间轮流切换
4.      "ACTION_ZERO": 0x21010C05,                #
   回零 0x21010C05 - 0 初始化机器人关节
5.      "ACTION_EMERGENCY_STOP": 0x21020C0E,      #
   软急停 0x21020C0E - 0 使机器人软急停
6.      "ACTION_FLATLAND_SLOW_WALK": 0x21010300,  #
   平地低速步态 0x21010300 - 0 使机器人从当前步态切换到低速步态
7.      "ACTION_FLATLAND_MEDIUM_WALK": 0x21010307,  #
   平地中速步态 0x21010307 - 0 使机器人从当前步态切换到中速步态
8.      "ACTION_FLATLAND_FAST_WALK": 0x21010303,  #
   平地高速步态 0x21010303 - 0 使机器人从当前步态切换到高速步态
9.      "ACTION_NORMAL_CRAWL": 0x21010406,        #
   正常/匍匐 0x21010406 - 0 使机器人从当前步态切换到匍匐低速行走步态,
   或从匍匐低速行走步态切至正常低速行走步态
```

10. "ACTION_GRASPING_OBSTACLE_WALK": 0x21010402, #
抓地越障步态 0x21010402 - 0 使机器人从当前步态切换到抓地步态

11. "ACTION_GENERAL_OBSTACLE_WALK": 0x21010401, #
通用越障步态 0x21010401 - 0 使机器人从当前步态切换到通用步态

12. "ACTION_HIGH_STEP_OBSTACLE_WALK": 0x21010407, #
高踏步越障步态 0x21010407 - 0 使机器人从当前步态切换到高踏步步态

13. "ACTION_TWIST_BODY": 0x21010204, #
扭身体 0x21010204 - 0 处于力控状态（静止站立）

14. "ACTION_ROLL_OVER": 0x21010205, #
翻身 0x21010205 - 0 趴下状态

15. "ACTION_MOONWALK": 0x2101030C, #
太空步 0x2101030C - 0 处于力控状态（静止站立）

16. "ACTION_BACKFLIP": 0x21010502, #
后空翻 0x21010502 - 0 趴下状态

17. "ACTION_GREET": 0x21010507, #
打招呼 0x21010507 - 0 趴下状态

18. "ACTION_JUMP_FORWARD": 0x2101050B, #
向前跳 0x2101050B - 0 趴下状态

19. "ACTION_TWIST_JUMP": 0x2101020D, #
扭身跳 0x2101020D - 0 处于力控状态（静止站立）

20.

21. # 原地模式

22. "ACTION_In_place_mode": 0x21010D05, #
原地模式: 0x21010D05 - 0

23.	"ACTION_Adjust_the_roll_angle": 0x21010131,	#
	调整横滚角: 0x21010131 0 [-12553,12553], 取正值时向右翻滚	
24.	"ACTION_Adjust_the_pitch_angle": 0x21010130,	#
	调整俯仰角: 0x21010130 0 [-6553,6553], 取正值时低头	
25.	"ACTION_Adjust_the_height_of_body": 0x21010102,	#
	调整身体高度: 0x21010102 0 [-20000,20000], 取正值时抬高身体	
26.	"ACTION_Adjust_the_yaw_angle": 0x21010135,	#
	调整偏航角: 0x21010135 0 [-9553,9553], 取正值时向右旋转	
27.		
28.	# 移动模式	
29.	"ACTION_Mobile_mode": 0x21010D06,	#
	移动模式: 0x21010D06 - 0	
30.	"ACTION_Translate_left_and_right": 0x21010131,	#
	左右平移: 0x21010131 0 [-12553,12553], 指定机器人 y 轴上的期望线速度, 正值向右	
31.	"ACTION_pan_back_and_forth": 0x21010130,	#
	前后平移: 0x21010130 0 [-6553,6553], 指定机器人 x 轴上的期望线速度, 正值向前	
32.	"ACTION_turn_left_and_right": 0x21010135,	#
	左右转弯: 0x21010135 0 [-9553,9553], 指定机器人的期望角速度, 正值向右转	
33.		
34.	# 自主模式	
35.	"Autonomous_mode": 0x21010C03,	#
	自主模式 0x21010C03 - 0 使机器人从手动模式切入自主模式	

```

36.
37.     # 手动模式
38.     "Manual_mode": 0x21010C02, #
    手动模式 0x21010C02 - 0 使机器人从自主模式切入手动模式
39.
40.     # 超声波雷达
41.     "radar": 0x21012109 #
    0x21012109 0x40 0 超声波有效量程为[0.28m,4.50m],
42. #
    当障碍物距离小于 0.28m 时显示为 0.28m,
43. #
    当障碍物距离大于 4.50m 时显示为 4.50m。
44. }

```

6.5. 示例：让机器狗站立

这是一个测试实例，通过简单的命令指令让机器狗站立，展示了如何通过简单的命令指令让机器狗站立。通过导入相关模块、建立通信链接、发送心跳包和发送站立指令，实现对机器狗的基本控制。为后续更复杂的操作提供了基础，验证了通信和控制指令的有效性。

```

1.  #!/usr/bin/env python
2.  # -*- coding: utf-8 -*-
3.
4.  # 导入定义的机器狗相关的包
5.  from sendcommand import SendToCommand, heartbeat #
    发送简单指令与心跳包

```

```

6.     from socketnetwork import network_utils                                     #
通信函数
7.     from command.udp_command import *                                         #
存放各种结构体与状态数据、指令
8.
9.     # 建立通讯
10.    sfd, target_address = network_utils.setup_socket_and_address()
11.
12.    # 建立心跳包线程，按照心跳函数中的频率发送心跳包
13.    heartbeat_thread = MyRepeatThread("HeartbeatThread", heartbeat.send_udp_heartbeat, 0.25, None, sfd, target_address)
14.    heartbeat_thread.start()
15.
16.    # 站立指令
17.    SendToCommand.perform_action(sfd, target_address, 0x21010202, 0, 0)

```

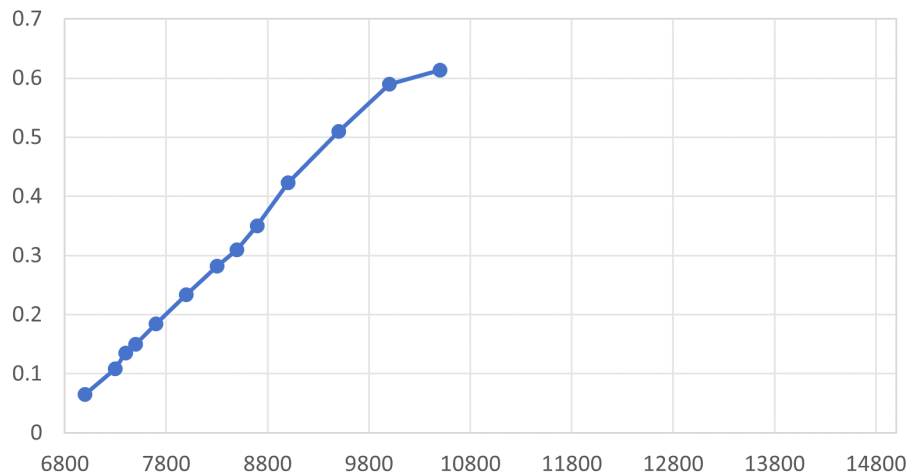
6.6. 建立速度模型

UDP 指令下发之后，对于狗的位移距离是根据 UDP 下发多长时间决定，而在相同的时间内，val 值的大小又会对狗的速度造成影响，所以没法精确控制狗的位移距离，如果是抽象的 val 值控制速度则更难，所以本实验提出了一种简单的思路来解决狗的控制问题，加入我们想要让狗位移 5 米，直接告诉他 5 米，他就会自己前进 5 米，这一切的基础是能得到狗的一种控制方式，这个方式叫做速度模型，下面给出速度模型的样例和构建思路。

long(m)	time(s)	每秒的平均	val
0.07	1	0.07	7000
0.12	2	0.06	7000
0.2	3	0.066666667	7000
0.1	1	0.1	7300
0.23	2	0.115	7300
0.33	3	0.11	7300
0.14	1	0.14	7400
0.28	2	0.14	7400
0.38	3	0.126666667	7400
0.15	1	0.15	7500
0.3	2	0.15	7500
0.45	3	0.15	7500
0.18	1	0.18	7700
0.38	2	0.19	7700

上表是采集到的数据具体含义（以第一行为例）：在 val 值给出 7000 的情况下，UDP 指令下发持续 1 秒钟，机器狗走了 0.07m；后面的数据依次类推，然后对所有的值进行大量的测试，包括横向平移，选择，可以得到如下图像（给出前进的图像，其他的方式与前进的测量方式完全相同，要注意一点，每个人测量的是不同）：

前进图



上图中 y 轴是速度，x 轴是对应的 val 值，在对曲线进行拟合可以得到方程组。得到拟合方程之后，我们可以做如下方式

```
1. def go_straight(long, speedgear=3):
2.     # 定义一个函数 go_straight, 接收两个参数: Long 和 speedgear,
      speedgear 有默认值 3
3.     # Long 代表距离, speedgear 代表档位
4.
5.     # 定义一个字典 speedgear_ditc, 键为档位, 值为对应的速度值 (单
      位是 rpm)
6.     speedgear_ditc = {
7.         1:7000, # 1 档对应 7000rpm
8.         2:7500, # 2 档对应 7500rpm
9.         3:8000, # 3 档对应 8000rpm
10.        4:8700, # 4 档对应 8700rpm
11.        5:9000, # 5 档对应 9000rpm
12.        6:10500, # 6 档对应 10500rpm
13.    }
```

```

14.
15.     # 根据传入的 speedgear 参数, 从字典中获取对应的速度值, 赋值
    给 val
16.     val = speedgear_ditc[speedgear]
17.
18.     # 如果传入的 Long 参数小于 0, 则将 val 取反
19.     if long < 0:
20.         val = -val
21.
22.     # 根据 val 计算速度 (单位是 m/s), 这里使用了二次方程
23.     speed_per_second_meter = (-1.5059e-
    08) * val ** 2 - (4.1944e-04) * val - 2.1804
24.
25.     # 计算所需时间, 使用 abs 确保时间是正数
26.     times = abs(long / speed_per_second_meter)
27.
28.     # 如果 Long 参数大于等于 0, 则使用另一组系数计算速度
29.     else:
30.         speed_per_second_meter = (-7.237e-
    09) * val ** 2 + 0.0002933 * val - 1.643
31.
32.     # 计算所需时间
33.     times = long / speed_per_second_meter
34.
35.     # 返回一个列表, 包含计算出的时间和速度
36.     return [times, val]

```

函数 go_straight:

目的：计算在给定档位下直线行驶特定距离所需的时间。

参数：

long：需要行驶的距离，单位米。

speedgear：车辆的档位，可选参数，默认值为 3。档位影响车辆的速度。

函数内部逻辑：

速度字典：函数内部定义了一个名为 `speedgear_ditc` 的字典，将每个档位映射到一个特定的速度值。

获取速度：根据提供的 `speedgear` 参数，从字典中获取对应的速度值，存储在变量 `val` 中。

距离的正负处理：

如果 `long` 小于 0，函数将 `val` 取反，这表示行驶方向相反。然后使用一个特定的二次方程来计算每秒钟可以行驶的距离 `speed_per_second_meter`。接着，计算所需时间 `times`，并通过 `abs` 函数取绝对值来确保时间是正数。

如果 `long` 大于或等于 0，使用另一个不同的二次方程来计算每秒钟可以行驶的距离，然后直接根据这个速度计算所需时间。

返回值：函数返回一个列表，包含两个元素：

times：根据距离和速度计算出的时间，表示行驶所需时间。

val：车辆的速度值，即在给定档位下的速度。

我们做同样的操作可以得到其他的方程的代码。

```
1. def translate_left_and_right(long, speedgear=3):
2.     # 档位速度只是参考值，具体速度按照实际来判断，默认速度是 3
   档
3.     speedgear_ditc = {
```

Error! Unknown document property name.

```

4.         1: 14000,
5.         2: 18000,
6.         3: 21000,
7.         4: 24000,
8.         5: 27000,
9.         6: 34000,
10.    }
11.    val = speedgear_ditc[speedgear]
12.    if long < 0:
13.        val = -val
14.        speed_per_second_meter = -(1.6e-05) * val -
0.2256
15.        times = abs(long / speed_per_second_meter)
16.    else:
17.        speed_per_second_meter = (1.517e-05) * val -
0.1748
18.        times = long / speed_per_second_meter
19.    return [times, val]

```

函数 `translate_left_and_right`:

这个函数接受两个参数：`long`（距离，单位米）和 `speedgear`（档位，可选，默认为 3）。

它首先定义了一个字典 `speedgear_ditc`，将档位映射到相应的速度值。

然后，根据给定的档位，函数从字典中获取速度值，并将其存储在变量 `val` 中。

如果 `long` 小于 0，函数将 `val` 取反，并使用一个线性方程来计算每秒钟可以行驶的距离，然后计算所需的时间。

如果 long 大于或等于 0,函数使用另一个不同的线性方程来计算每秒钟可以行驶的距离,然后计算所需的时间。

函数返回一个列表,包含所需的时间和速度值。

```
1. def revolve_left_and_right(angle):
2.     def find_closest_output(input_value):
3.         # 判断角度是否超过了360度,超过了则需要则算掉
4.         if input_value > 360:
5.             input_value = input_value / (input_value % 360)
6.
7.         # 输出到输入的映射字典
8.         output_to_input_map = {
9.             0: 0,
10.            15: 0.1,
11.            30: 0.2,
12.            45: 0.53,
13.            60: 0.6,
14.            75: 0.9,
15.            90: 1.3,
16.            105: 1.5,
17.            120: 1.9,
18.            135: 2.2,
19.            150: 2.3,
20.            165: 2.5,
21.            167: 2.8,
22.            185: 2.9,
23.            195: 3.1,
24.            210: 3.4,
```

```

25.         225: 3.7,
26.         240: 4,
27.         255: 4.3,
28.         270: 4.5,
29.         285: 4.7,
30.         300: 5,
31.         315: 5.3,
32.         330: 5.6,
33.         345: 5.8,
34.         360: 5.9,
35.     }
36.     # 获取所有输出键并将其转换为列表
37.     keys = list(output_to_input_map.keys())
38.     # 寻找与输入值最接近的输出键
39.     closest_key = min(keys, key=lambda x: abs(x -
input_value))
40.     # 返回与最接近键关联的输入值
41.     return output_to_input_map[closest_key]
42.
43.
44.     times = find_closest_output(abs(angle)) # 获取时间与
角度的关系
45.     if angle < 0:
46.         val = -10000
47.     else:
48.         val = 10000
49.
50.     return [times, val]

```

函数 `revolve_left_and_right`:

这个函数接受一个参数 `angle` (角度, 单位是度)。

它定义了一个嵌套函数 `find_closest_output`, 用于找到与给定角度最接近的输出值。这个嵌套函数首先检查角度是否超过 360 度, 如果是, 则将其除以 360 度取余数以调整角度值。然后, 它使用一个字典 `output_to_input_map` 来映射角度到输出值, 并找到与输入角度最接近的映射值。

主函数 `revolve_left_and_right` 使用 `find_closest_output` 函数来获取与绝对值角度对应的时间。

如果 `angle` 小于 0, 函数将变量 `val` 设置为 -10000, 否则设置为 10000。

函数返回一个列表, 包含时间 (通过 `find_closest_output` 函数计算) 和 `val`。

6.7. 线程模板

线程模板是用于控制机器狗动作的一个基类, 所有需要发送的指令将使用这个类的方法, 不同指令的发送间隔时间可以根据具体情况进行调整。

```
1. #!/usr/bin/env python
2. # 这是一个 shebang 行, 告诉操作系统使用哪个解释器来执行这个脚本,
   这里是使用环境变量中定义的python 解释器。
3.
4. # -*- coding: utf-8 -*-
5. # 指定文件的编码格式为 utf-8。
6.
7. # 导入所需的模块
8. import threading
9. # 导入 Python 的线程模块, 用于多线程编程。
10.
```

```
11. import time
12. # 导入时间模块，用于处理时间相关的功能。
13.
14. import logging
15. # 导入日志模块，用于记录日志信息。
16.
17. # 从其他模块导入特定的类或函数
18. from speeds.sportspeed import *
19. # 从 speeds 模块的 sportspeed 子模块中导入所有类和函数。
20.
21. from sendcommand import SendToCommand, heartbeat
22. # 从 sendcommand 模块中导入 SendToCommand 类和 heartbeat 函数。
23.
24. from socketnetwork import network_utils
25. # 从 socketnetwork 模块中导入 network_utils。
26.
27. # 定义一个继承自 threading.Thread 的线程类 MyRepeatThread
28. class MyRepeatThread(threading.Thread):
29.     # 这是一个自定义线程类，继承自 threading.Thread。
30.
31.     def __init__(self, name, action, interval, time_limit
        =None, *args) -> None:
32.         # 构造函数，初始化线程的名称、动作、间隔时间、最大时间
        阈值和参数。
33.         super(MyRepeatThread, self).__init__()
34.         self.name = name
```

```

35.         self.action = action
36.         self.interval = interval # 发送命令的频率
37.         self.time_limit = time_limit # 最大时间阈值
38.         self.args = args
39.         self.stopped = threading.Event() # 用于停止线程的
        事件
40.         self.start_time = time.time() # 记录线程开始的时间
41.         self.global_var = 0
42.         self.current_time = time.time() # 当前时间
43.         self.current_time_start = 0
44.
45.         # 线程运行的方法
46.         def run(self) -> None:
47.             print(f"Starting {self.name}")
48.             while not self.stopped.is_set():
49.                 self.current_time = time.time()
50.                 # 如果超过时间阈值，则停止线程
51.                 if self.time_limit is not None and self.check
                    _time_and_stop(time.time()):
52.                     break
53.                 try:
54.                     self.action(*self.args) # 执行动作，展开参
                        数
55.                 except KeyboardInterrupt:
56.                     self.stopped = True
57.                 finally:
58.                     pass

```

```

59.          # 计算需要休眠多长时间以保持固定频率
60.          action_start_time = time.time()
61.          elapsed_time = action_start_time -
            self.current_time
62.          time_to_wait = max(0, self.interval -
            elapsed_time)
63.          time.sleep(time_to_wait) # 休眠以保持固定频率
64.
65. self.stopped.set() # 确保线程停止状态被设置
66. logging.info(f'离开线程: {self.name}')
67. # 记录日志, 表示线程已退出。
68.
69. # 检查是否超过时间阈值并停止线程的方法
70. def check_time_and_stop(self, current_time) -> bool:
71.     # 定义一个方法来判断当前时间是否超过了线程开始时间加上时间
        阈值。
72.
73.     if current_time - self.start_time > self.time_limit:
74.         logging.info(f'{self.name} 由于超过时间阈值
            {self.time_limit}秒, 系统自动停止!')
75.         # 如果当前时间减去线程开始时间大于时间阈值, 则记录日志
            并设置停止事件。
76.         self.stopped.set()
77.         return True
78.     elif self.global_var == 1:
79.         self.stopped.set()

```

Error! Unknown document property name.

```

80.         return True
81.         # 如果 global_var 变量为 1, 也设置停止事件并返回 True。
82.         return False
83.
84. # 停止线程的方法
85. def stop(self) -> None:
86.     self.stopped.set()
87.     # 定义一个方法来设置停止事件, 用于外部调用以停止线程。
88.
89. # 打印对象所有属性及其值的方法
90. def print_attributes(self):
91.     self.current_time_start = time.time() -
        self.start_time
92.     # 计算并更新当前时间与线程开始时间的差值。
93.     return {attr: value for attr, value in vars(self).items()}
94.     # 返回一个字典, 包含对象的所有属性名和它们的值。
95.
96. # 定义前进动作的函数, 返回一个线程实例
97. def action_go_straight(long, speedgear=3) -> MyRepeatThread:
98.
99.     # 定义一个函数, 用于创建前进动作的线程实例。
100.
101.     # Long 参数表示前进的距离, speedgear 表示速度档位, 默认为
        3。
100.     times, val = go_straight(long, speedgear)
101.     # 调用 go_straight 函数计算需要执行的次数和值。

```

```

102.     ACTION_pan_back_and_forth_thread = MyRepeatThread(
103.         "ACTION_pan_back_and_forth_thread",
104.         SendToCommand.perform_action, 0.1, times, sfd, ta
            rget_address, 0x21010130, val, 0
105.     )
106.     # 创建并返回一个MyRepeatThread 线程实例，用于执行前进动作。
107.     # 不直接启动线程
108.
109. # 定义左右平移动作的函数，返回一个线程实例
110. def action_turn_left_and_right(long, speedgear=3) -> MyRe
    peatThread:
111.     # 定义一个函数，用于创建左右平移动作的线程实例。
112.     times, val = translate_left_and_right(long, speedgear)
113.     # 调用translate_left_and_right 函数计算需要执行的次数和
        值。
114.     ACTION_turn_left_and_right_thread = MyRepeatThread(
115.         "ACTION_turn_left_and_right_thread",
116.         SendToCommand.perform_action, 0.1, times, sfd, ta
            rget_address, 0x21010131, val, 0
117.     )
118.     # 创建并返回一个MyRepeatThread 线程实例，用于执行左右平移
        动作。
119.     # 不直接启动线程
120.
121. # 定义左右旋转动作的函数，返回一个线程实例

```

```

122. def action_revolve_left_and_right(angle) -> MyRepeatThrea
    d:
123.     # 定义一个函数，用于创建左右旋转动作的线程实例。
124.     # angle 参数表示旋转的角度。
125.     times, val = revolve_left_and_right(angle)
126.     # 调用revolve_left_and_right 函数计算需要执行的次数和值。
127.     ACTION_revolve_left_and_right = MyRepeatThread(
128.         "ACTION_revolve_left_and_right",
129.         SendToCommand.perform_action, 0.1, times, sfd, ta
            rget_address, 0x21010135, val, 0
130.     )
131.     # 创建并返回一个MyRepeatThread 线程实例，用于执行左右旋转
        动作。
132.     # 不直接启动线程
133.
134. # 定义一个字典，将动作编号映射到相应的动作函数
135. actions_dict = {
136.     1: action_go_straight,
137.     2: action_turn_left_and_right,
138.     3: action_revolve_left_and_right,
139.     # 其他函数根据实际情况添加
140. }
141. # 这个字典用于根据动作编号快速找到对应的动作函数，方便动态调用。

```

机器狗所有的需要发送的指令都将使用这个类的方法，比如发送的间隔时间可以根据指令来判断不同代码的解释如下：

1. 导入模块

Error! Unknown document property name.

代码首先导入了 Python 标准库中的 `threading`、`time` 和 `logging` 模块，这些模块分别用于线程操作、时间处理和日志记录。此外，还导入了三个自定义模块，这些模块包含了与特定硬件或应用相关的功能。

2. 日志配置

使用 `logging.basicConfig(level=logging.DEBUG)` 配置了日志系统，这意味着所有等级为 `DEBUG` 及以上的日志信息都会被记录。

3. 网络设置

`network_utils.setup_socket_and_address()` 函数用于设置网络通信所需的套接字和目标地址。这个函数的具体实现没有在代码中给出，但它负责创建一个网络连接，用于发送和接收数据。

4. 自定义线程类 `MyRepeatThread`

这个类继承自 `threading.Thread`，是一个自定义线程类，用于执行重复的动作。它具有以下特点：

初始化：在构造函数中，除了标准的线程属性外，还可以设置动作函数、执行间隔、最大执行时间以及一些额外的参数。

运行逻辑：`run` 方法包含了线程的主要执行逻辑。它在一个循环中执行动作函数，并根据需要休眠以保持固定的动作频率。

时间检查：`check_time_and_stop` 方法用于检查是否应该停止线程，这是由于达到了最大执行时间或某个全局变量被设置。

停止方法：`stop` 方法用于外部控制线程停止。

属性打印：`print_attributes` 方法用于打印线程的所有属性，这对于调试和监控线程状态非常有用。

5. 动作执行函数

代码定义了三个函数 `action_go_straight`、`action_turn_left_and_right` 和 `action_revolve_left_and_right`，这些函数用于创建和返回 `MyRepeatThread` 的实例。每个函数都封装了一个特定的动作：

前进：`action_go_straight` 函数计算前进的距离，并创建一个线程来执行这个动作。

平移：`action_turn_left_and_right` 函数计算平移的距离，并创建一个线程来执行平移动作。

旋转：`action_revolve_left_and_right` 函数计算旋转的角度，并创建一个线程来执行旋转动作。

这些函数都调用了自定义模块中的函数（如 `go_straight`、`translate_left_and_right` 和 `revolve_left_and_right`）来获取执行动作所需的参数。

6. 动作字典 `actions_dict`

这个字典将整数键映射到对应的动作函数，这是用于根据输入选择执行哪个动作。例如，如果输入是 1，就会执行 `action_go_straight` 函数。

7. 其他细节

异常处理：在 `run` 方法中，有一个 `try-except` 块来捕获 `KeyboardInterrupt` 异常，这通常是用户尝试停止程序时触发的。

休眠计算：在执行动作后，代码计算了动作执行所需的时间，并根据这个时间来决定线程下一次休眠的时间，以保持固定的动作频率。

6.8. 后台输入动作序列

后台输入动作序列主要实现了一个用于用户输入和处理机器狗动作序列的

系统。用户通过输入一系列动作 ID 和相应的参数，系统将根据这些输入生成并更新动作序列，并提示用户为每个动作提供具体参数。

```
1. # 定义一个字典，将动作编号映射到相应的动作描述
2. actions_name = {
3.     1: '直走 (0.3 表示前进 0.3 米, -0.3 表示后退 0.3 米) ',
4.     2: '平移 (0.3 表示右平移 0.3 米, -0.3 表示左平移 0.3 米) ',
5.     3: '旋转 (45 表示向右旋转 45 度, -45 表示向左旋转 45 度) '
6. }
7.
8. # 初始化一个空字典，用于存储动作参数
9. actions_params = {}
10.
11. # 初始化一个空列表，用于存储动作序列
12. actions_sequence = []
13.
14. # 初始化三个空列表，分别用于存储每个动作编号对应的参数值
15. val1, val2, val3 = [], [], []
16.
17. # 打印提示信息，告知用户接下来将进入输入环节
18. print("下面即将进入到输入环节，根据你输入的数字和参数索引，机器
    狗将会做出相应运动")
19. print("输入格式：动作 ID, 参数索引，例如：3,2,1,2,3,1（依次用英
    文逗号分隔开）")
20.
21. # 从用户那里接收动作 ID 输入，使用逗号分隔
```

```

22. x = input("请输入动作 ID(回车结束) : ")
23. # 将输入的字符串分割成列表, 移除空字符串
24. x = [i for i in x.split(',') if i != '']
25.
26. # 假设总是使用参数列表中的第一个参数, 并将其添加到动作序列中
27. for action_id in x:
28.     actions_sequence.append((int(action_id), 0))
29.
30. # 定义一个函数, 用于更新动作序列
31. def update_actions_sequence(actions_sequence):
32.     # 初始化计数器
33.     count_1 = 0
34.     count_2 = 0
35.     count_3 = 0
36.
37.     # 结果列表
38.     updated_sequence = []
39.
40.     # 遍历原始序列
41.     for action in actions_sequence:
42.         if action[0] == 3:
43.             # 如果是3, 更新3的计数器, 并添加到结果列表
44.             updated_sequence.append((3, count_3))
45.             count_3 += 1
46.         elif action[0] == 1:
47.             # 如果是1, 更新1的计数器, 并添加到结果列表
48.             updated_sequence.append((1, count_1))

```

Error! Unknown document property name.

```

49.         count_1 += 1
50.         elif action[0] == 2:
51.             # 如果是2, 更新2 的计数器, 并添加到结果列表
52.             updated_sequence.append((2, count_2))
53.             count_2 += 1
54.     # 返回更新后的动作序列
55.     return updated_sequence
56.
57. # 更新动作序列
58. actions_sequence = update_actions_sequence(actions_sequence)
59.
60. # 提示用户可以设置速度档位
61. logging.info('这里可以设置速度档位, 比如在显示前进的时候可以输入 0.3,6 意思是以 6 档的速度前进 0.3 米')
62.
63. # 遍历更新后的动作序列, 并打印每个动作的名称
64. for index, action in enumerate(actions_sequence):
65.     action_name = actions_name.get(action[0], "未知动作")
66.     print(f'第{index+1}个动作是 : {action_name}')
67.
68. # 接收用户对于每个动作的具体参数输入
69. for index, action in enumerate(actions_sequence):
70.     vals = input('你想要做的操作是 : ')
71.
72.     # 检查输入是否包含逗号

```

```

73.     if ', ' in vals:
74.         # 如果包含逗号, 则将输入按逗号分割, 并转换为浮点数
75.         vals = tuple(float(val.strip()) for val in vals.s
           plit(', '))
76.     else:
77.         # 如果没有逗号, 直接将输入转换为单个浮点数
78.         vals = (float(vals),)
79.
80.     # 打印参数值
81.     print(vals)
82.
83.     # 根据动作编号, 将参数值添加到对应的列表中
84.     if action[0] == 1:
85.         val1.append(vals)
86.     elif action[0] == 2:
87.         val2.append(vals)
88.     elif action[0] == 3:
89.         val3.append(vals)
90.
91.     # 将 val1, val2, val3 列表中的参数值分别作为值, 动作编号作为键,
       存储到 actions_params 字典中
92.     actions_params = {
93.         1: val1,
94.         2: val2,
95.         3: val3,
96.     }

```

深入分析这段代码的每一部分, 并详细解释其功能。

1. 定义动作和参数

```
actions_name = {  
    1: '直走 (0.3 表示前进 0.3 米, -0.3 表示后退 0.3 米) ',  
    2: '平移 (0.3 表示右平移 0.3 米, -0.3 表示左平移 0.3 米) ',  
    3: '旋转 (45 表示向右旋转 45 度, -45 表示向左旋转 45 度) '  
}
```

这部分定义了一个字典 `actions_name`，它将动作 ID 映射到对应的动作描述。动作 ID 为 1、2、3，分别对应直走、平移和旋转。

2. 初始化变量

```
actions_params = {}  
actions_sequence = []  
val1, val2, val3 = [], [], []
```

这里初始化了几个变量：

`actions_params`：一个空字典，用于存储每个动作的参数。

`actions_sequence`：一个空列表，用于存储动作序列。

`val1`、`val2`、`val3`：三个空列表，分别用于存储动作 1、2、3 的参数。

3. 用户输入动作 ID

```
print("下面即将进入到输入环节...")  
  
x = input("请输入动作 ID(回车结束): ")
```

这部分代码提示用户输入一系列动作 ID，这些 ID 通过逗号分隔。用户输入后，按回车结束输入。

4. 处理用户输入

```
x = x.split(',')  
x = [i for i in x if i != ""]
```

```
for action_id in x:
```

```
    actions_sequence.append((int(action_id), 0)) # 假设总是使用参数列表中的  
第一个参数
```

这里将用户的输入字符串分割成列表，然后去除空字符串，并转换为整数。接着，将每个动作 ID 与索引 0（假设总是使用第一个参数）一起添加到 `actions_sequence` 列表中。

5. 更新动作序列

```
def update_actions_sequence(actions_sequence):
```

```
    # ...
```

```
    return updated_sequence
```

```
actions_sequence = update_actions_sequence(actions_sequence)
```

定义了一个函数 `update_actions_sequence` 来更新动作序列。这个函数通过遍历 `actions_sequence` 并为每个动作类型增加一个计数器来更新序列。然后，调用这个函数来更新 `actions_sequence`。

6. 提示设置速度挡位

```
logging.info('这里可以设置速度挡位...')
```

这里使用 `logging` 模块记录了一条信息，提示用户可以设置速度挡位。

7. 遍历动作序列并获取参数

```
for index, action in enumerate(actions_sequence):
```

```
    # ...
```

```
    vals = input('你想要做的操作是: ')
```

```
    # ...
```

这部分代码遍历更新后的动作序列，对于每个动作，提示用户输入相应的操作参数。

8. 处理和存储参数

```
if ',' in vals:
    # ...
else:
    # ...
if action[0] == 1:
    val1.append(vals)
elif action[0] == 2:
    val2.append(vals)
elif action[0] == 3:
    val3.append(vals)
```

这里根据用户输入的参数，将其存储在相应的列表 `val1`、`val2` 或 `val3` 中。如果用户输入包含逗号，则将其分割并转换为浮点数；如果不包含逗号，则直接转换为浮点数。

9. 更新 `actions_params` 字典

```
actions_params = {
    1: val1,
    2: val2,
    3: val3,
}
```

最后，将 `val1`、`val2` 和 `val3` 列表分别作为值存储在 `actions_params` 字典中，键为对应的动作 ID。

6.9. 执行动作

通过遍历预先生成的动作序列，获取相应的动作函数和参数，并在机器狗上执行这些动作。通过用户输入的动作序列和参数，系统能够动态地选择并执行对

应的动作，确保机器狗按照预期进行移动、旋转等操作。同时，通过日志记录和参数验证，确保动作执行的安全性和可追溯性。

```
1. # 执行动作
2. for action_id, params_index in actions_sequence:
3.     # 通过动作ID从 actions_dict 字典中获取对应的动作函数，如果没有找到则返回None
4.     action_func = actions_dict.get(action_id, None)
5.
6.     if action_func:
7.         # 如果找到了动作函数，从 actions_params 字典中获取指定动作ID和参数索引对应的参数
8.         params = actions_params[action_id][params_index]
9.
10.        # 记录详细的动作执行信息到日志
11.        logging.info('-----
-----
-----')
12.        logging.info(f'action_func: {action_func}')
13.        logging.info(f'params: {params}')
14.
15.        # 检查params是否为元组类型
16.        if isinstance(params, tuple):
17.            # 如果是元组，调用动作函数并传入参数，使用*操作符将元组中的元素作为独立的参数传递
18.            thread = action_func(*params) # 正确# 展开参数列表并传递给函数
```

```

19.          # 启动线程
20.          thread.start()
21.          # 等待线程执行完成
22.          thread.join()
23.          # 打印线程的属性信息, 这里假设 thread 对象有一个
            print_attributes 方法
24.          pprint(thread.print_attributes())
25.          # 休眠 1 秒, 是为了在执行下一个动作之前提供短暂的延
            迟
26.          time.sleep(1)
27.          else:
28.          # 如果 params 不是元组或列表类型, 则记录错误日志
29.          logging.error('params 必须是一个 tuple 或者 列
            表')

```

这段代码是模拟机器狗执行一系列动作的过程。逐步分析代码的功能和结构:

1. 遍历动作序列

```
for action_id, params_index in actions_sequence:
```

这里使用一个循环遍历 `actions_sequence` 列表, 该列表包含了一系列的动作 ID 和参数索引。

2. 获取动作函数

```
action_func = actions_dict.get(action_id, None)
```

通过 `actions_dict` 字典根据动作 ID 获取对应的动作函数。如果找不到对应的动作 ID, 则 `action_func` 为 `None`。

3. 检查动作函数是否存在

```
if action_func:
```

如果存在动作函数，即 `action_func` 不是 `None`，则执行以下操作。

4. 获取动作参数

```
params = actions_params[action_id][params_index]
```

从 `actions_params` 字典中获取对应动作 ID 和参数索引的参数。

5. 记录日志信息

```
logging.info('-----')
-----')
logging.info(f'action_func: {action_func}')
logging.info(f'params: {params}')
```

使用 `logging` 模块记录动作函数和参数的信息。

6. 检查参数类型

```
if isinstance(params, tuple):
```

检查获取的参数是否为元组（`tuple`）类型。

7. 调用动作函数

```
thread = action_func(*params) # 展开参数列表并传递给函数
thread.start()
thread.join()
```

如果参数是元组类型，那么将参数展开并传递给动作函数 `action_func`。这里假设 `action_func` 是一个可以创建线程的函数。创建线程后，调用 `start()` 方法启动线程，然后调用 `join()` 方法等待线程执行完成。

8. 打印线程属性

```
pprint(thread.print_attributes())
```

这里使用了 `pprint` 函数（是 `prettyprint` 的简写，但未在前面的代码中定义）来打印线程的属性。同样，`print_attributes` 方法也未在前面的代码中定义，这里

假设它存在。

9. 暂停执行

```
time.sleep(1)
```

在执行完一个动作后，使用 `time.sleep(1)` 暂停 1 秒，是为了模拟动作之间的间隔。

10. 错误处理

```
else:
```

```
logging.error('params 必须是一个 tuple 或者 列表')
```

如果参数不是元组类型，记录一条错误日志。

第七章 运行示例

如果要运行代码，直接输入

```
python precise_control_main.py
```

然后根据提示，输入相应的值即可，如下所示：

```
(base) root@davinci-mini:/opt/lab/PreciseControl# python precise_control_main.py
Starting HeartbeatThread
下面即将进入到输入环节，根据你输入的数字和参数索引，机械狗将会做出相应运动
输入格式：动作ID,参数索引，例如：3,2,1,2,3,1（依次用英文逗号分隔开）
请输入动作ID(回车结束)：1,2,3
INFO:root:这里可以设置速度档位，比如在显示前进的时候可以出入 0.3,6 意思是以6档的速度前进0.3米
第1个动作是：直走（0.3表示前进0.3米，-0.3表示后退0.3米）
你想要做的操作是：0.3
(0.3,)
第2个动作是：平移（0.3表示右平移0.3米，-0.3表示左平移0.3米）
你想要做的操作是：0.3
(0.3,)
第3个动作是：旋转（45表示向右旋转45度，-45表示向左旋转45度）
你想要做的操作是：30
(30.0,)
{1: [(0.3,)], 2: [(0.3,)], 3: [(30.0,)]}
INFO:root:-----
INFO:root:action_func: <function action_go_straight at 0xe7fff5498280>
INFO:root:params: (0.3,)
Starting ACTION_pan_back_and_forth_thread
INFO:root:ACTION_pan_back_and_forth_thread由于超过时间阈值1.2487928335941945秒，系统自动停止！
INFO:root:离开线程：ACTION_pan_back_and_forth_thread
INFO:root:-----
INFO:root:action_func: <function action_turn_left_and_right at 0xe7fff527fee0>
INFO:root:params: (0.3,)
Starting ACTION_turn_left_and_right_thread
INFO:root:ACTION_turn_left_and_right_thread由于超过时间阈值2.0866662029630656秒，系统自动停止！
INFO:root:离开线程：ACTION_turn_left_and_right_thread
INFO:root:-----
INFO:root:action_func: <function action_revolve_left_and_right at 0xe7fff527ff70>
INFO:root:params: (30.0,)
Starting ACTION_revolve_left_and_right
INFO:root:ACTION_revolve_left_and_right由于超过时间阈值0.2秒，系统自动停止！
INFO:root:离开线程：ACTION_revolve_left_and_right
```

意思为第一步直走 0.3 米，第二步右平移 0.3 米，第三步右旋转 30 度。

第八章 附录

8.1. 腿部动力学建模

8.1.1. 拉格朗日方程

拉格朗日动力学方程是处理多自由度动力学问题的重要理论基础。对于处于完整约束并且约束力满足虚功原理的机械系统,可以使用拉格朗日动力学方程来描述系统随时间的变化。达朗贝尔原理指出,对于理想情况下具有两面约束的非自由质点系,在任一瞬时时刻,作用于该质系的主动动力及惯性力在质点系任意虚位移上所作的元功之和为零。可以表示为:

$$\sum_{i=1}^k (F_i - m_i \ddot{r}_i) \delta r_i = 0 \quad (2.1)$$

式中, r_i, m_i 分别表示质点 $P_i (i = 1, \dots, k)$ 的矢径和质量, F_i 则表示质点系统受到的主动力。

由于虚拟约束 δr_i 并不相互独立, 需要选择广义坐标来将方程(2.1) 转换为关于独立广义坐标的表达式。设 $q_j (j = 1, \dots, n)$ 为所研究质点的广义坐标, 则可将矢径 r_i 表示成关于广义坐标的形式:

$$\delta r_i = \sum_{j=1}^n \frac{\partial r_i}{\partial q_j} \delta q_j \quad (2.2)$$

因此, 根据式(2.1), 式(2.2)中左端第二项可变为可得:

$$\sum_{i=1}^k m_i \ddot{r}_i \delta r_i = \sum_{i=1}^k \sum_{j=1}^n m_i \ddot{r}_i \frac{\partial r_i}{\partial q_j} \delta q_j \quad (2.3)$$

使用微分运算可将上式重新整理得到:

$$\sum_{i=1}^k m_i \ddot{r}_i \frac{\partial r_i}{\partial q_j} = \sum_{i=1}^k \frac{d}{dt} \left(m_i \dot{r}_i \frac{\partial r_i}{\partial q_j} \right) - \sum_{i=1}^k m_i \dot{r}_i \frac{d}{dt} \left(\frac{\partial r_i}{\partial q_j} \right) \quad (2.4)$$

接下来, 根据公式:

$$\frac{\partial \dot{\mathbf{r}}_i}{\partial \dot{q}_j} = \frac{\partial \mathbf{r}_i}{\partial q_j} \#(2.5)$$

$$\frac{d}{dt} \left(\frac{\partial \mathbf{r}_i}{\partial \dot{q}_j} \right) = \frac{\partial \dot{\mathbf{r}}_i}{\partial \dot{q}_j} \#(2.6)$$

$$K = \sum_{i=1}^k \frac{1}{2} m_i \dot{\mathbf{r}}_i^2 \#(2.7)$$

式(2.7)可变为:

$$\sum_{i=1}^k m_i \ddot{\mathbf{r}}_i \frac{\partial \mathbf{r}_i}{\partial \dot{q}_j} = \frac{d}{dt} \frac{\partial K}{\partial \dot{q}_j} - \frac{\partial K}{\partial q_j} \#(2.8)$$

将式(2.8)带入式(2.1)可得:

$$\sum_{j=1}^n \left(\sum_{i=1}^k \mathbf{F}_i \frac{\partial \mathbf{r}_i}{\partial \dot{q}_j} - \frac{d}{dt} \frac{\partial K}{\partial \dot{q}_j} + \frac{\partial K}{\partial q_j} \right) \delta q_j = 0 \#(2.9)$$

由于广义坐标相互独立,同时令第 j 个广义力为:

$$\psi_j = \sum_{i=1}^k \mathbf{F}_i \frac{\partial \mathbf{r}_i}{\partial \dot{q}_j} \#(2.10)$$

引入势能函数 $P(q)$,则外界施加的广义力可表示为主动动力 $\mathbf{F}_i$ 引起的广义力

ψ_j 与势能 $P(q)$ 引起的广义力之和:

$$\tau_j = \psi_j + \frac{\partial P(q)}{\partial \dot{q}_j} \#(2.11)$$

根据式子(2.10)和(2.11),公式(2.9)可被写为下列形式:

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{q}_j} \right) - \frac{\partial \mathcal{L}}{\partial q_j} = \tau_j \#(2.12)$$

式中, $\mathcal{L} = K - P$ 为拉格朗日算子,定义为系统的动能减去势能,化简后得到:

$$\frac{d}{dt} \left(\frac{\partial K}{\partial \dot{q}_j} \right) - \frac{\partial K}{\partial q_j} + \frac{\partial P}{\partial q_j} = \tau_j \#(2.13)$$

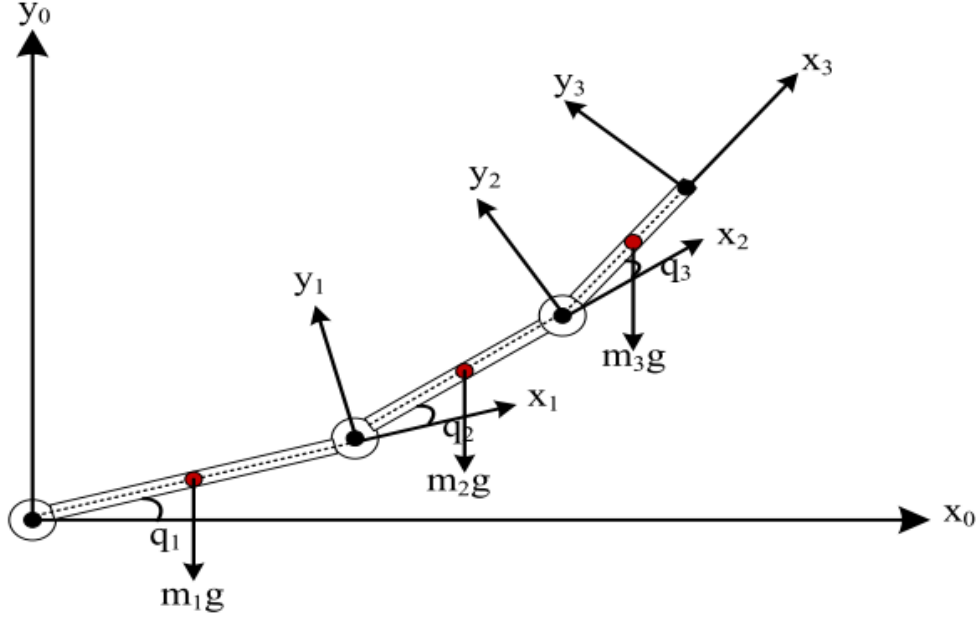
式(2.13)为拉格朗日动力学方程。

8.1.2. 动力学模型

将拉格朗日方程写成腿的动力学方程的表达式:

$$\mathbf{M}(q)\ddot{\mathbf{q}} + \mathbf{C}(q,\dot{\mathbf{q}}) + \mathbf{G}(q) = \boldsymbol{\tau}$$

在三关节手指建模中， $\tau = [\tau_1 \ \tau_2 \ \tau_3]^T$ 是控制力的矩阵， $\ddot{q} = [\ddot{q}_1 \ \ddot{q}_2 \ \ddot{q}_3]^T$ 是角加速度向量， $M(q) \in \mathbb{R}^{3 \times 3}$ 为惯性矩阵， $C(q, \dot{q}) \in \mathbb{R}^{3 \times 1}$ 是哥式力和离心力的结合向量， $G(q) \in \mathbb{R}^{3 \times 1}$ 是重力矩阵



图中，对于 $i = 1, 2, 3$ ， q_i 表示关节转角，也作为广义坐标； m_i 表示手指指节的质量； l_i 表示指节的长度； l_{ci} 表示前一个关节与指节 i 的质心之间的距离。

此处使用 Denavit-Hartenberg 关节变量作为广义坐标，从而可得到表达式：

$$v_n^0 = J_v \dot{q} \quad (2.14)$$

$$w_n^0 = J_w \dot{q}$$

式中， v_n^0 代表灵巧手指质心的线速度， w_n^0 代表机器狗指质心角速度， n 代表关节数量。 J_v 表示各关节对应的雅可比矩阵， $\dot{q}$ 表示各关节转动的角速度。

根据柯尼希定理，动能等于杆件质心的平动动能和杆件相对于质心的转动动能之和，则系统总动能

$$K = \frac{1}{2} m v^T v + \frac{1}{2} \omega^T I \omega$$

$$I = RIR^T$$

I 是附体坐标系的惯性张量矩阵， I 是惯性坐标系中的惯性张量矩阵，将（下面式子带入上面式子）得到：

$$K = \frac{1}{2} \dot{q}^T \left[\sum_{i=1}^n \{ m_i J_{vi}^T J_{vi} + J_{\omega i}^T R_i I_i R_i^T J_{\omega i} \} \right] \dot{q} = \frac{1}{2} \dot{q}^T M \dot{q}$$

得到 M 为：

$$M(q) = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix}$$

科里奥利矩阵经过计算可得：

$$c_{kj} = \sum_{i=1}^n c_{ijk}(q) \dot{q}_i = \sum_{i=1}^n \frac{1}{2} \left\{ \frac{\partial m_{kj}}{\partial q_i} + \frac{\partial m_{ki}}{\partial q_j} - \frac{\partial m_{ij}}{\partial q_k} \right\} \dot{q}_i$$

$$c_{ijk} = \frac{1}{2} \left\{ \frac{\partial d_{ki}}{\partial q_j} + \frac{\partial d_{kj}}{\partial q_i} - \frac{\partial d_{ij}}{\partial q_k} \right\}$$

将下面式子带入上面式子得到：

$$C(q, \dot{q}) = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}$$

灵巧手指的重力势能等于三根指节的势能之和，通过每根指节质量乘以重力加速度以及质心高度计算得到：

$$P = P_1 + P_2 + P_3$$

通过对重力势能微分可得重力向量矩阵：

$$g(q) = \begin{bmatrix} g_1 \\ g_2 \\ g_3 \end{bmatrix}$$

因此，由以上公式可推导出机器狗腿动力学模型为：

$$M(q)\ddot{q} + C(q, \dot{q}) + G(q) = \tau$$

系统动力学模型简化为：

$$\ddot{q} = D(q)^{-1}(\tau - C(q, \dot{q})\dot{q} - g(q))$$

令 $x_1(t) = q, x_2(t) = \dot{q}$ ，那么机器狗腿动力学模型状态空间表达式表示为：

$$\begin{cases} \dot{x}_1(t) = x_2(t) \\ \dot{x}_2(t) = D(x_1(t))^{-1}(\tau - \mathcal{C}(x_1(t), x_2(t))x_2(t) - g(x_1(t))) \\ y(t) = x_1(t) \end{cases}$$